

Breadth First Search

2026.04.07

Depth First Search

Uniform Search – Dijkstra Algorithm

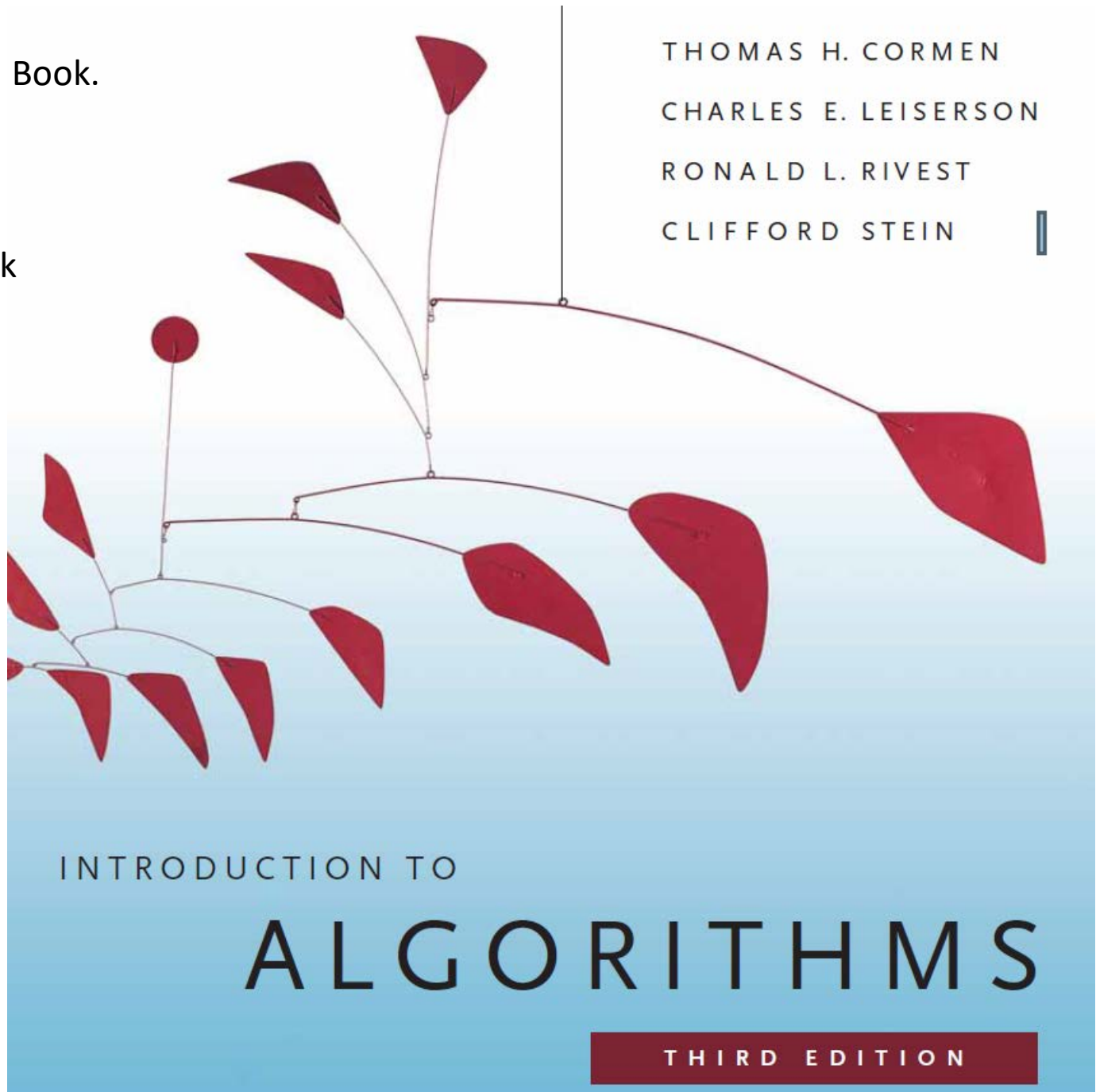
Chris Ding

This PPT is exclusively for students taking Prof. Ding's class. Not permitted to pass on to others.

Algorithm Book.

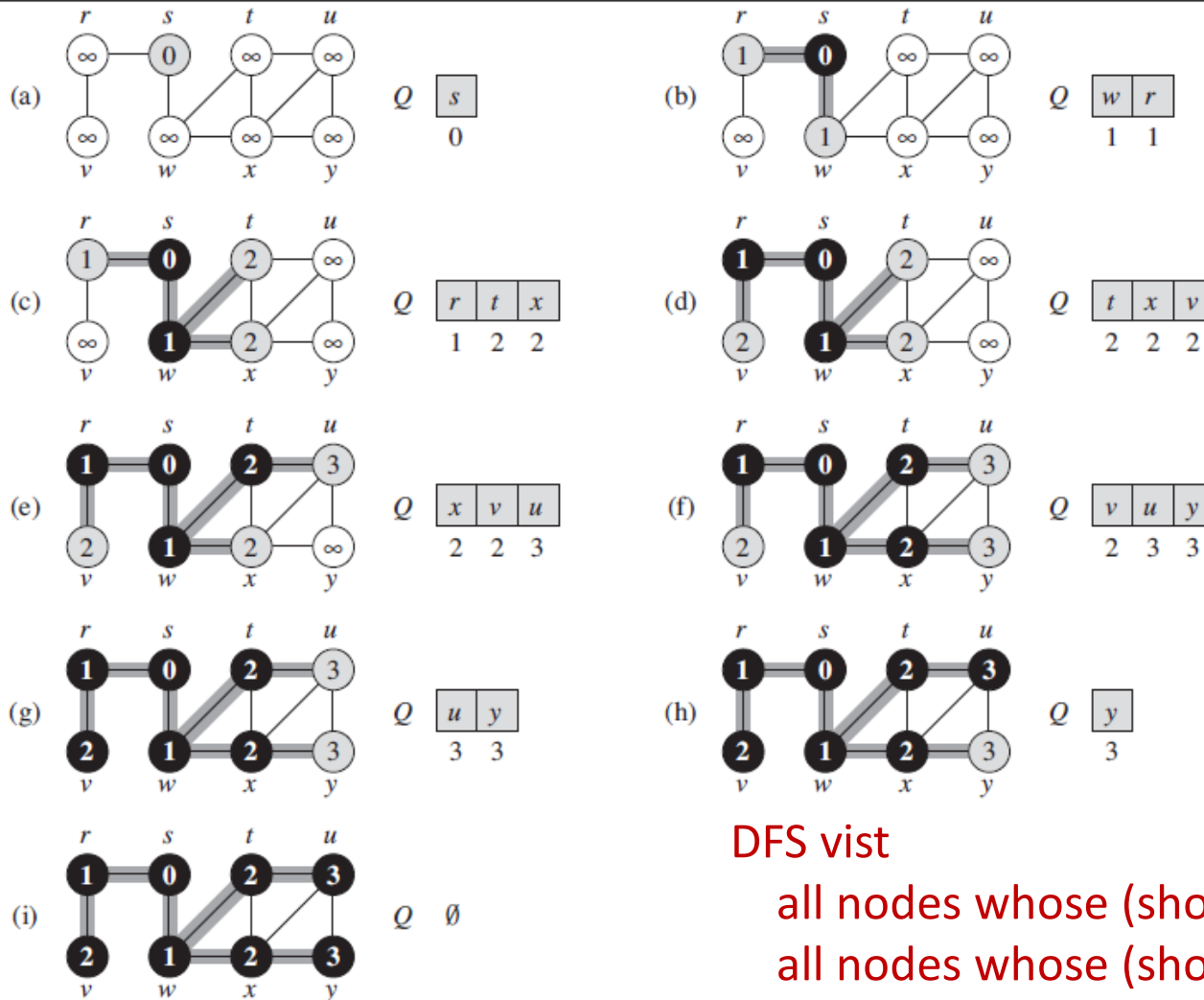
The most popular book
in computer science

THOMAS H. CORMEN
CHARLES E. LEISERSON
RONALD L. RIVEST
CLIFFORD STEIN



BFS on a given/known graph [Moore, 1959; Lee, 1961]

code and example



BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = WHITE$ 
3       $u.d = \infty$ 
4       $u.\pi = NIL$ 
5   $s.color = GRAY$ 
6   $s.d = 0$ 
7   $s.\pi = NIL$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = DEQUEUE(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == WHITE$ 
14              $v.color = GRAY$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = BLACK$ 

```

DFS vist

all nodes whose (shortest distance from s) = 1

all nodes whose (shortest distance from s) = 2

.....

Search algorithm generates a search tree

3.3.1 Infrastructure for search algorithms

Search algorithms require a data structure to keep track of the search tree that is being constructed. For each node n of the tree, we have a structure that contains four components:

- n .STATE: the state in the state space to which the node corresponds;
- n .PARENT: the node in the search tree that generated this node;
- n .ACTION: the action that was applied to the parent to generate the node;
- n .PATH-COST: the cost, traditionally denoted by $g(n)$, of the path from the initial state to the node, as indicated by the parent pointers.

function CHILD-NODE(*problem*, *parent*, *action*) **returns** a node

return a node with

STATE = *problem*.RESULT(*parent*.STATE, *action*),

PARENT = *parent*, ACTION = *action*,

PATH-COST = *parent*.PATH-COST + *problem*.STEP-COST(*parent*.STATE, *action*)

function GRAPH-SEARCH(*problem*) **returns** a solution, or failure

 initialize the frontier using the initial state of *problem*

initialize the explored set to be empty

loop do

if the frontier is empty **then return** failure

 choose a leaf node and remove it from the frontier

Different way of choosing LEADS to different algorithm

if the node contains a goal state **then return** the corresponding solution

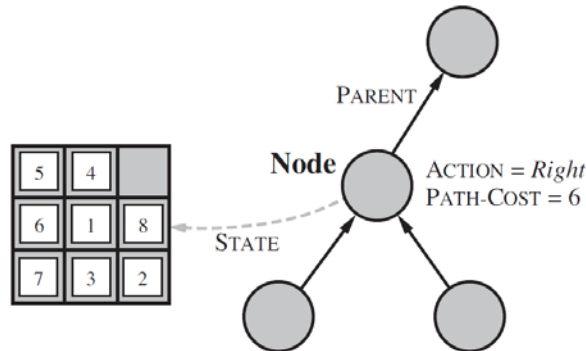
add the node to the explored set

 expand the chosen node, adding the resulting nodes to the frontier

do actions, find adjacent nodes

only if not in the frontier or explored set

currently in processing or already processed



Expand a node using one action, generate one adjacent node, all information on the node are updated

Expand the node using all available actions generate all adjacent nodes

Figure 3.10 Nodes are the data structures from which the search tree is constructed. Each has a parent, a state, and various bookkeeping fields. Arrows point from child to parent.

function BREADTH-FIRST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

frontier $\leftarrow$ a FIFO queue with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the shallowest node in *frontier* */

add *node*.STATE to *explored*

for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

if *child*.STATE is not in *explored* or *frontier* **then**

if *problem*.GOAL-TEST(*child*.STATE) **then return** SOLUTION(*child*)

frontier $\leftarrow$ INSERT(*child*, *frontier*)

add to the queue

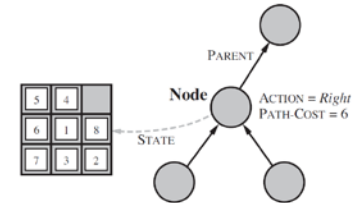


Figure 3.10 Nodes are the data structures from which the search tree is constructed. Each has a parent, a state, and various bookkeeping fields. Arrows point from child to parent.

function BREADTH-FIRST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

frontier $\leftarrow$ a FIFO queue with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the shallowest node in *frontier* */

 add *node*.STATE to *explored*

for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

if *child*.STATE is not in *explored* or *frontier* **then**

if *problem*.GOAL-TEST(*child*.STATE) **then return** SOLUTION(*child*)

frontier $\leftarrow$ INSERT(*child*, *frontier*)

function UNIFORM-COST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

frontier $\leftarrow$ a priority queue ordered by PATH-COST, with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the lowest-cost node in *frontier* */

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

 add *node*.STATE to *explored*

for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

if *child*.STATE is not in *explored* or *frontier* **then**

frontier $\leftarrow$ INSERT(*child*, *frontier*)

else if *child*.STATE is in *frontier* with higher PATH-COST **then**

 replace that *frontier* node with *child*

function UNIFORM-COST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

frontier $\leftarrow$ a priority queue ordered by PATH-COST, with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the lowest-cost node in *frontier* */

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

 add *node*.STATE to *explored*

for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

if *child*.STATE is not in *explored* or *frontier* **then**

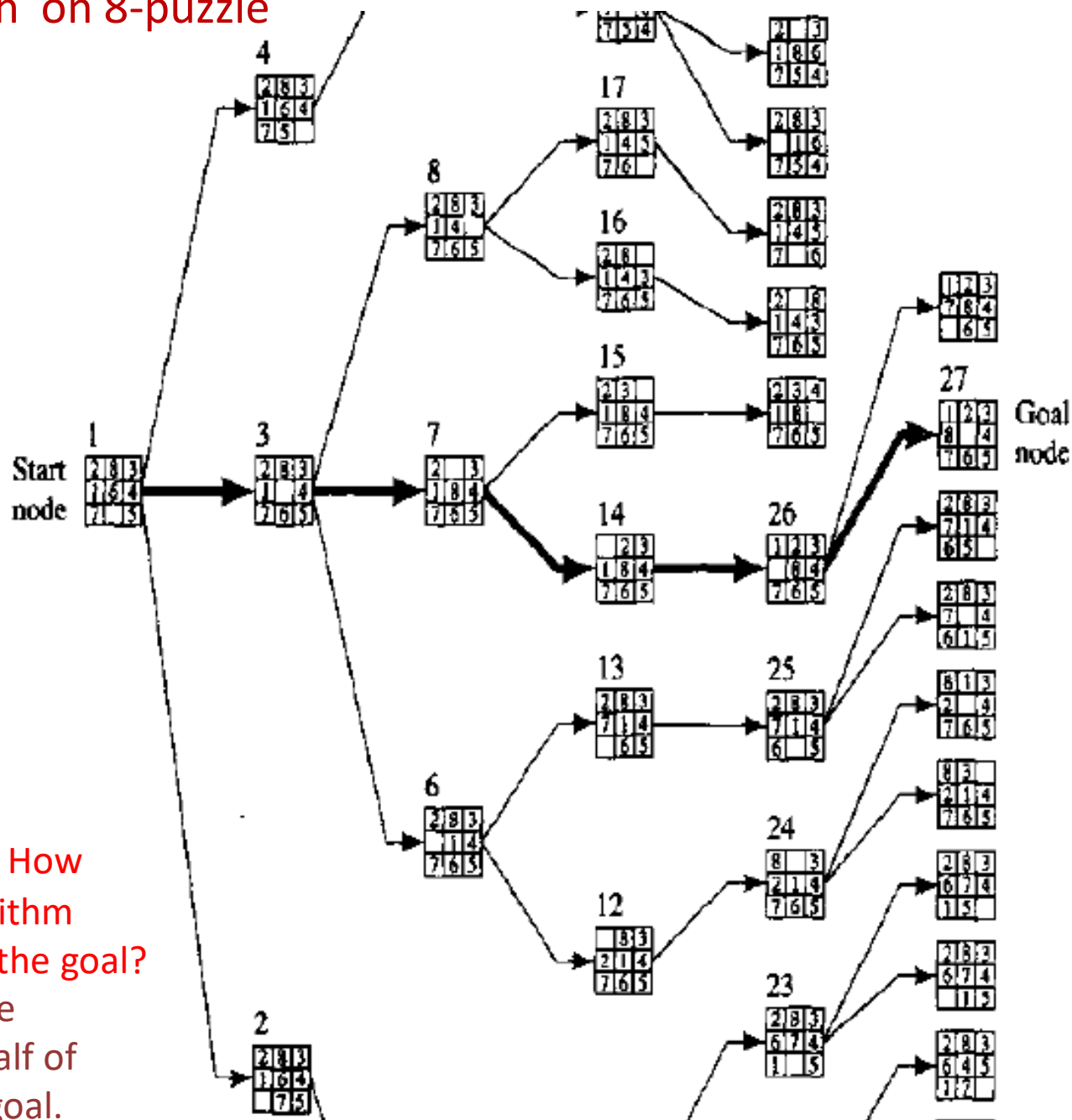
frontier $\leftarrow$ INSERT(*child*, *frontier*)

else if *child*.STATE is in *frontier* with higher PATH-COST **then**

 replace that *frontier* node with *child*

The existing state has a higher cost
Replace the node with new parent and cost

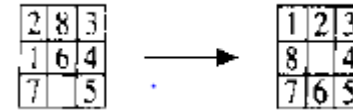
Breadth-First Search on 8-puzzle



Homework-BFS1: How many states the algorithm have searched to get the goal?

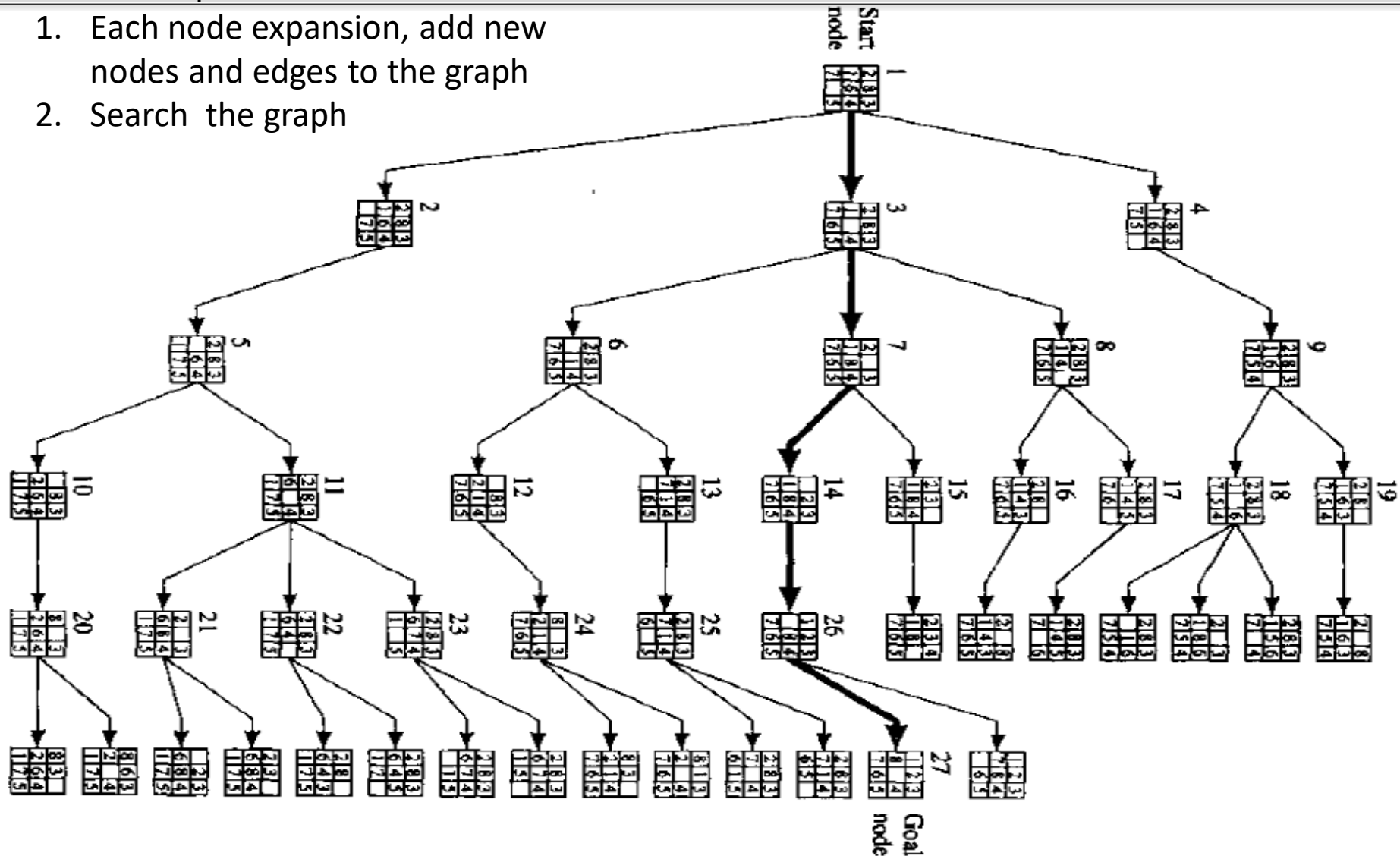
Assume on level 5, the algorithm searched half of the states to get the goal.

Breadth-First Search on 8-puzzle



AI Node Expansion and Search:

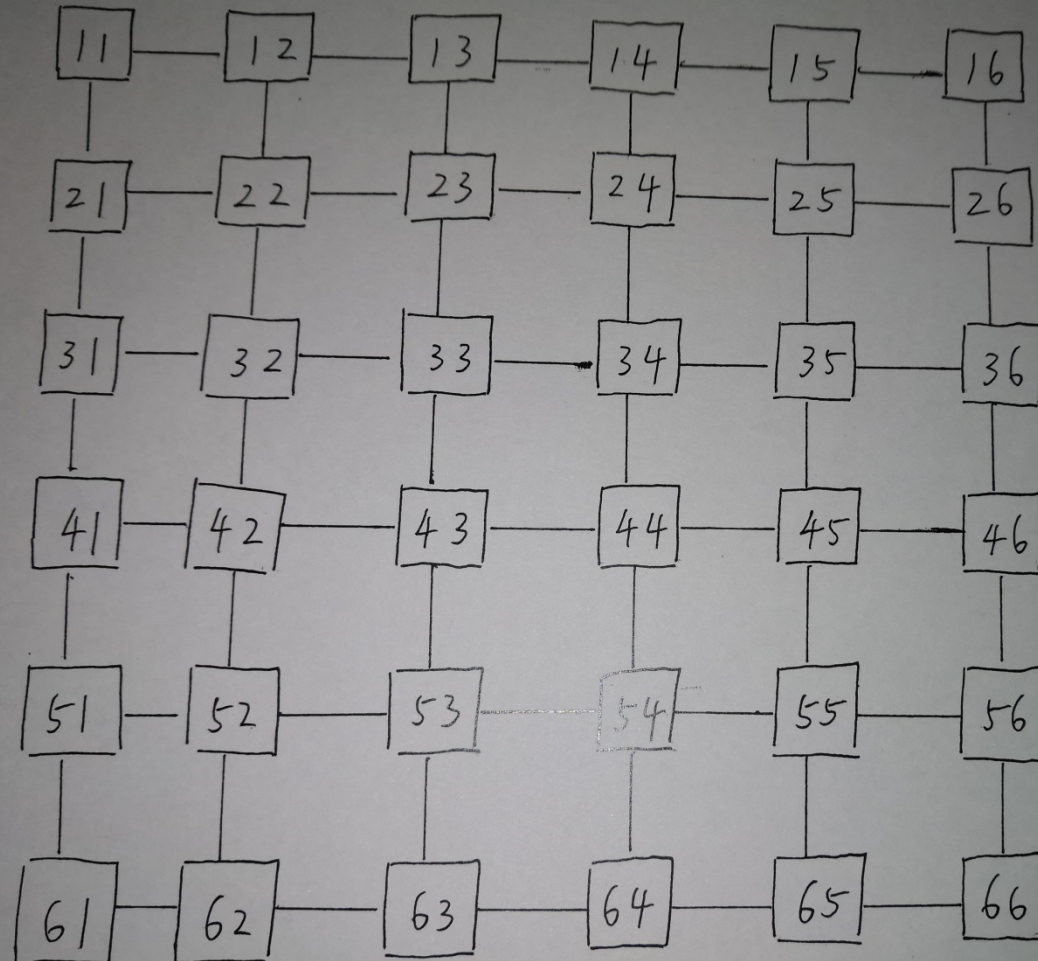
1. Each node expansion, add new nodes and edges to the graph
2. Search the graph





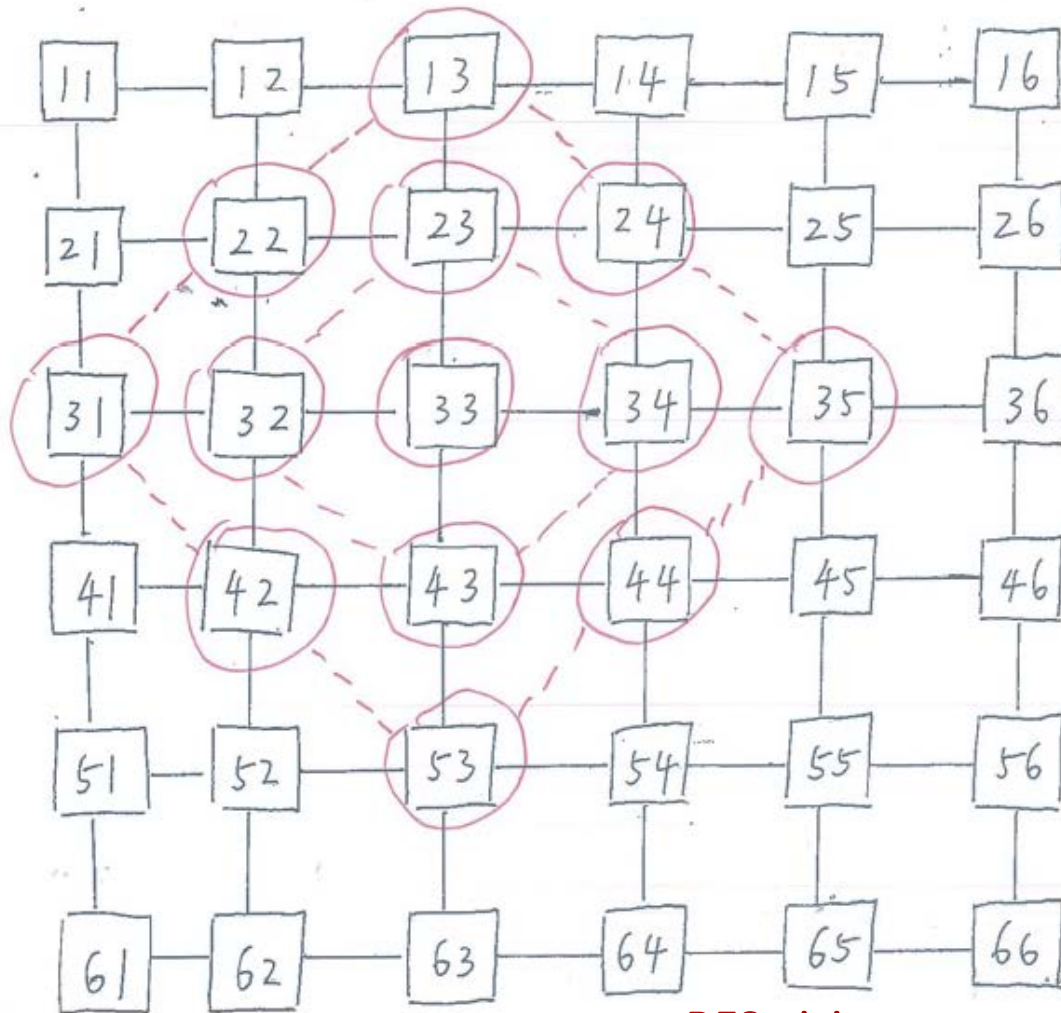
Assume the travel suitcase locker uses 2 digits
Each digit can be (1,2,3,4,5,6)

This is the global graph with all nodes and edges



BFS on a 2-digit locker graph

BFS



Beginning from the start node s here is $(3,3)$, BFS visits its closest nodes first, visit the 2nd closest nodes, 3rd closest nodes, etc.

BFS visit

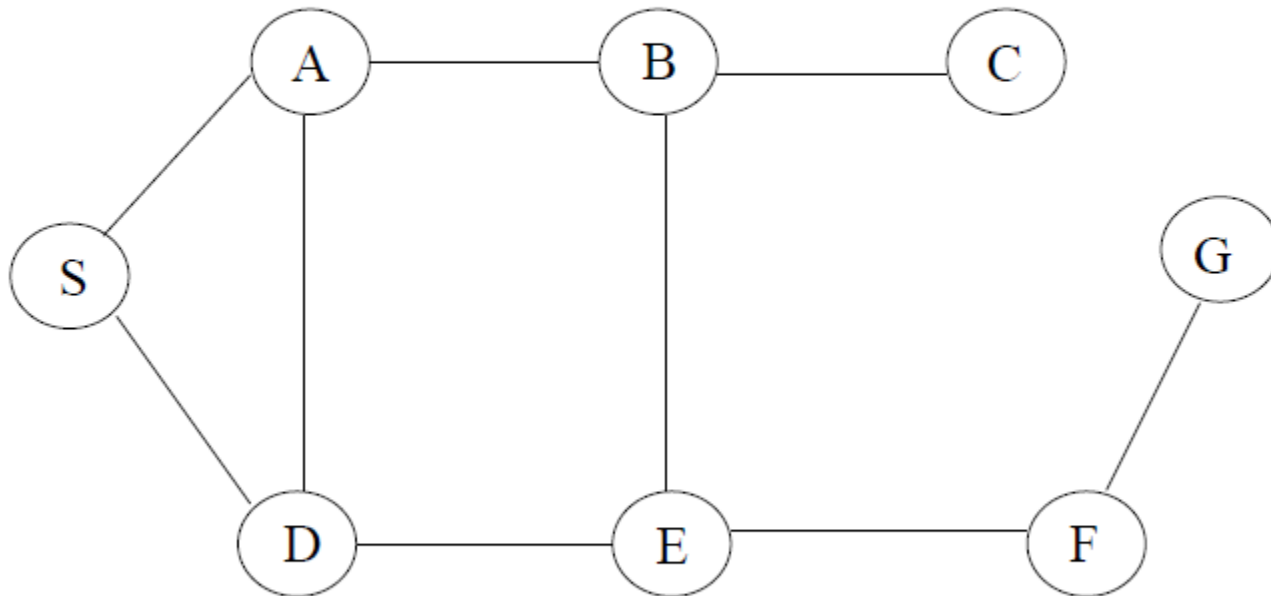
all nodes (whose shortest distance from s) = 1

all nodes (whose shortest distance from s) = 2

The locker graph is a graph. The BFS search-path is a tree.

Breadth First Search (BFS)

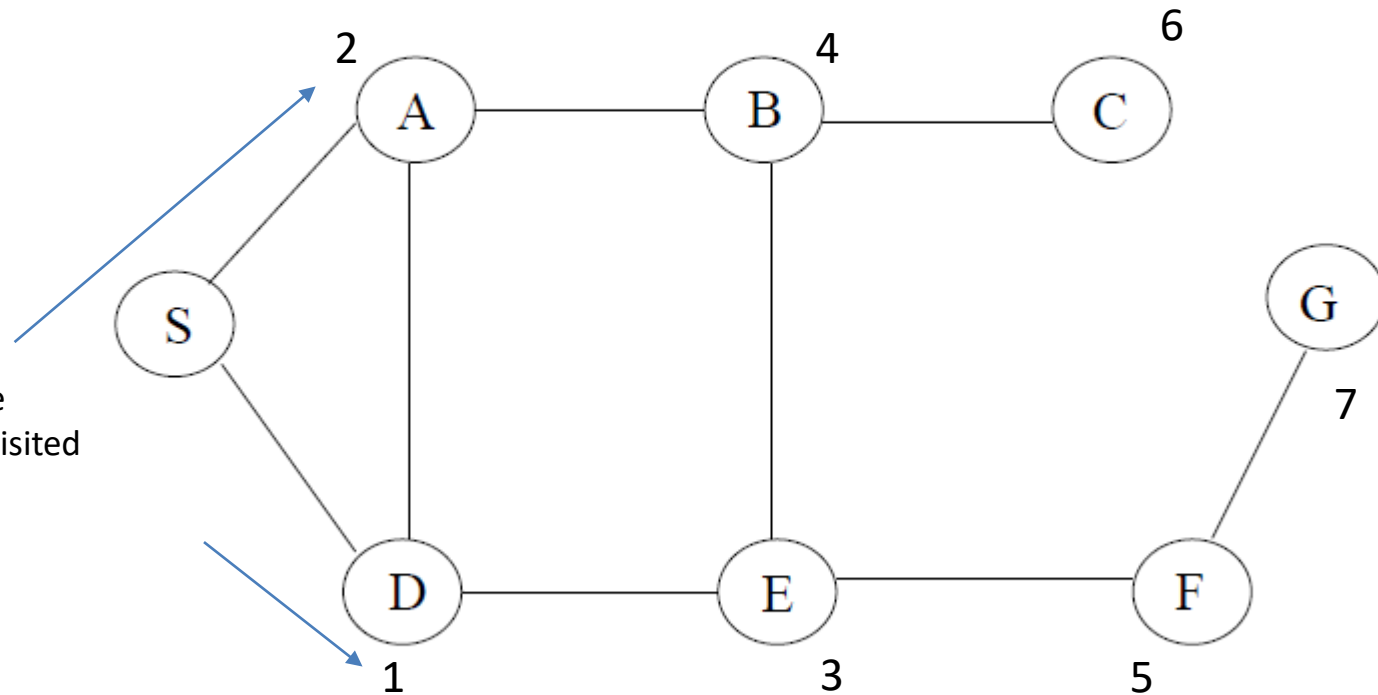
Example: Map Navigation



S = start, G = goal, other nodes = intermediate states, links = legal transitions

Breadth First Search (BFS)

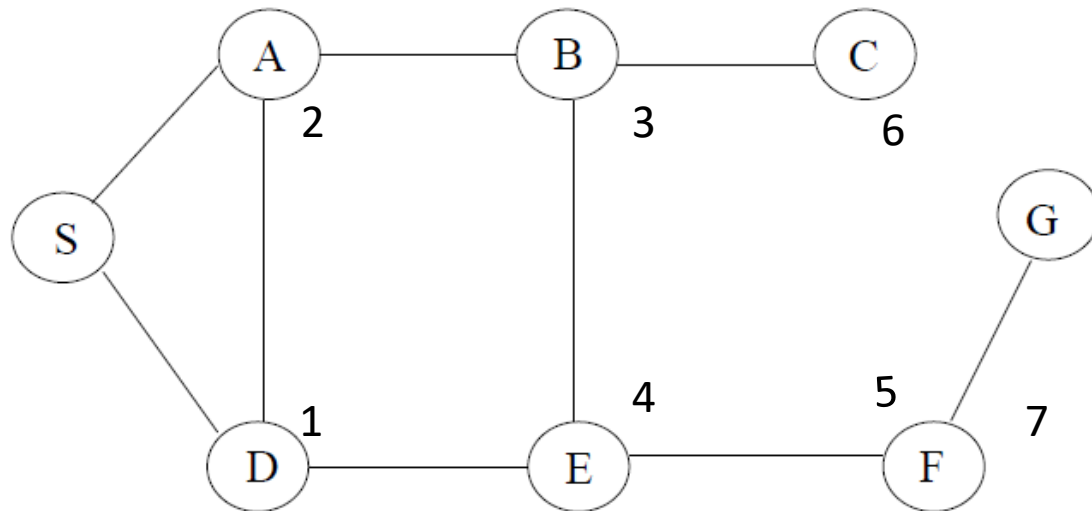
Example: Map Navigation



S = start, G = goal, other nodes = intermediate states, links = legal transitions

Breadth First Search (BFS)

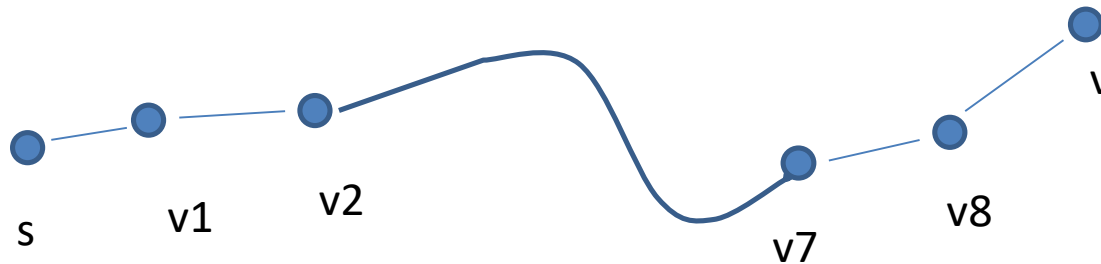
Example: Map Navigation



S = start, G = goal, other nodes = intermediate states, links = legal transitions

Question: Starting from S, which node will be visited next, node is D not A?

BFS finds the correct shortest path on the graph



Any sub part of a shortest path is itself a shortest path. Why?

Theorem (*Correctness of breadth-first search*)

Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$. Then, during its execution, BFS discovers every vertex $v \in V$ that is reachable from the source s , and upon termination, $v.d = \delta(s, v)$ for all $v \in V$. Moreover, for any vertex $v \neq s$ that is reachable from s , one of the shortest paths from s to v is a shortest path from s to $v.\pi$ followed by the edge $(v.\pi, v)$.

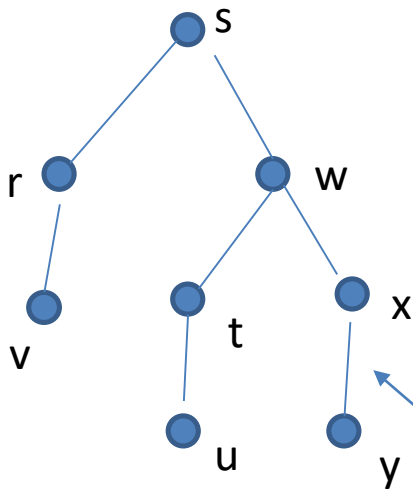
Intuitively, DFS vist

all nodes (with shortest distance from s) = 1

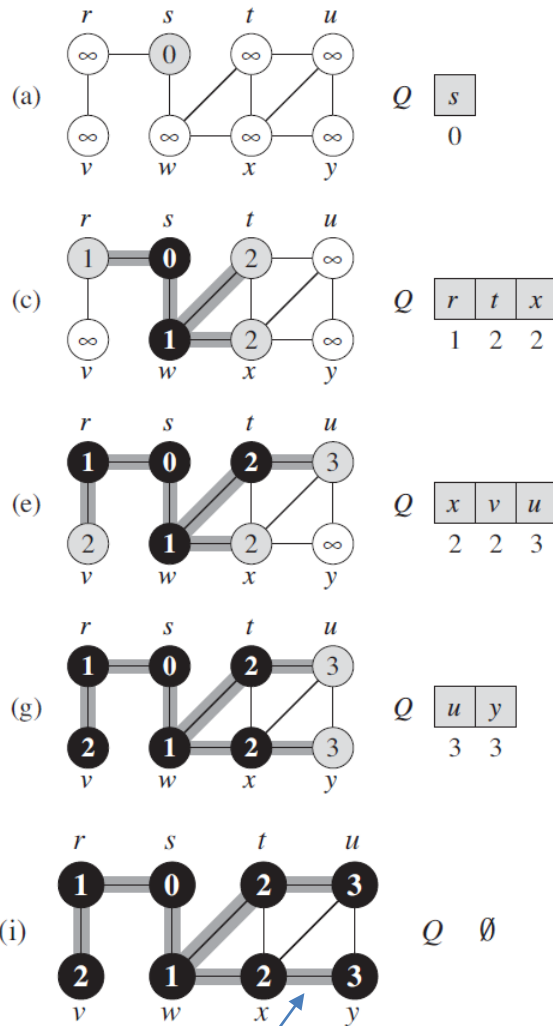
all nodes (with shortest distance from s) = 2

... ..

Bread First Search Tree



Tree edge



Note: many edges in the graph do not show up in BFS tree.
Why?

BFS correctly mark shortest **distance** from **S** to all other vertices.

But we want the shortest **path** from **S** to **V** .

The following procedure traces back from **V** , using **V** 's predecessor repeatedly, to the starting vertices.

The following procedure prints out the vertices on a shortest path from s to v , assuming that BFS has already computed a breadth-first tree:

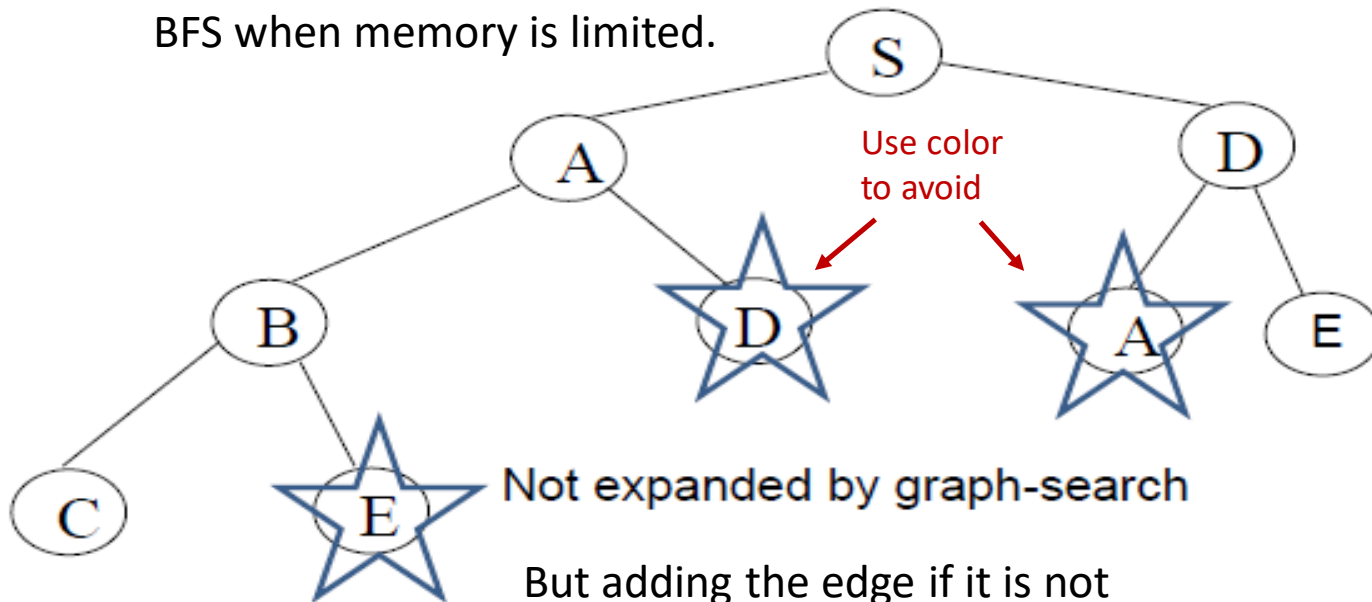
```
PRINT-PATH( $G, s, v$ )
1  if  $v == s$ 
2      print  $s$ 
3  elseif  $v.\pi == \text{NIL}$ 
4      print “no path from”  $s$  “to”  $v$  “exists”
5  else PRINT-PATH( $G, s, v.\pi$ )
6      print  $v$ 
```

This procedure runs in time linear in the number of vertices in the path printed, since each recursive call is for a path one vertex shorter.

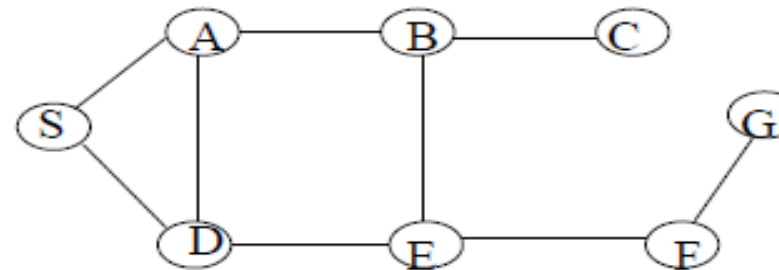
Breadth First Search (BFS) with node expansion

Breadth-First Search Graph

BFS when memory is limited.



But adding the edge if it is not already in the expanded graph



Search strategies

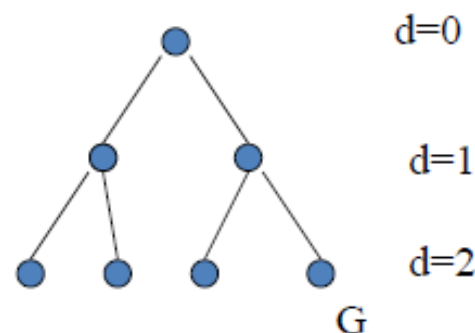
- A search **strategy** is defined by picking the **order of nodes**
- Strategies are evaluated along the following dimensions:
 - **completeness**: does it always find a solution if one exists?
 - **time complexity**: number of nodes generated
 - **space complexity**: maximum number of nodes in memory
 - **optimality**: does it always find a least-cost solution?
- Time and space complexity are measured in terms of
 - b : maximum branching factor of the search tree
 - d : depth of the least-cost solution
 - m : maximum depth of the state space (may be ∞)

Complexity of Breadth-First Search

- Time Complexity**

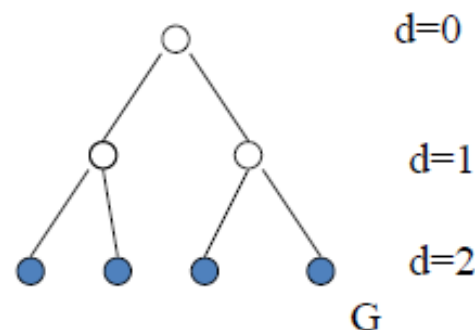
- assume (worst case) that there is 1 goal leaf at the RHS
- so BFS will expand all nodes

$$= 1 + b + b^2 + \dots + b^d$$
$$= O(b^d)$$



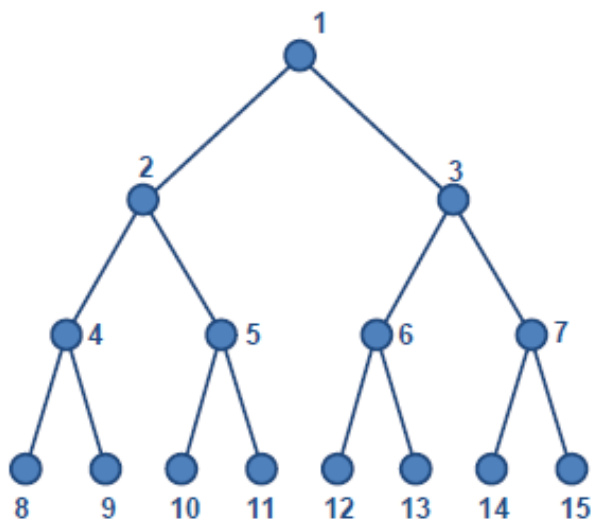
- Space Complexity**

- how many nodes can be in the queue (worst-case)?
- at depth d there are b^d unexpanded nodes in the Q = $O(b^d)$



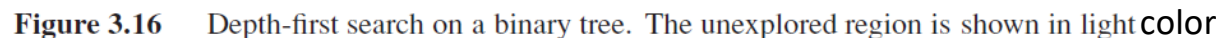
Breadth-First Search (BFS) Properties

- Solution Length: optimal
- Expand each node once (can check for duplicates, performs graph-search)
- Search Time: $O(b^d)$
- Memory Required: $O(b^d)$
- Drawback: requires exponential space



You go straight visiting nodes you encounter until you hit a wall,
then you turn around and try another route (adjacent vertex),
then **backup, try another route (adjacent vertex), ...**

After visit H , you hit the wall, **backup** to G , visit I



Depth-First Search

This is the main routine. Once you are on a vertex, visit one of its adjacent vertex in turn if it is not already visited (color gray or black). It is nested (recursively called).

depth-first-search

- mark vertex as visited (gray color)

- for each adjacent vertex

 - if unvisited

 - do a depth-first search on adjacent vertex

Note: the recursive calling of “depth-first-search”. This is the DFS-visit subroutine.

Depth-First Search

You go straight to visit nodes you encounter until you hit a wall/deadend, then you backtrack, turn around and try another route, ...

depth-first-search

- mark vertex as visited (gray color)

- for each adjacent unvertex

 - if unvisited

 - do a depth-first search on adjacent vertex

Note: the recursive calling of “depth-first-search”. This is the DFS-visit subroutine.

Pseudo-Code for Depth-First Search

depth-first-search

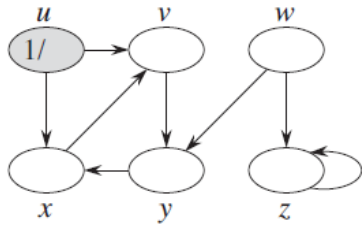
mark vertex as visited

for each adjacent vertex

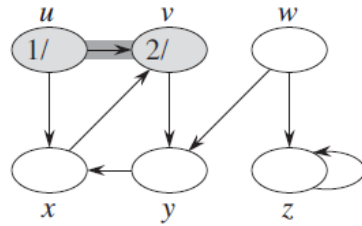
if unvisited

do a depth-first search on adjacent vertex

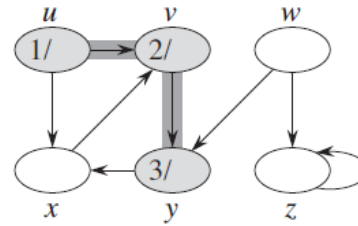
DFS code and example



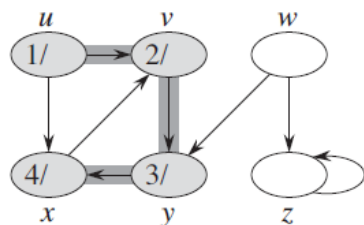
(a)



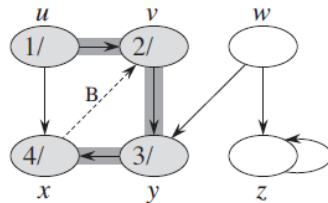
(b)



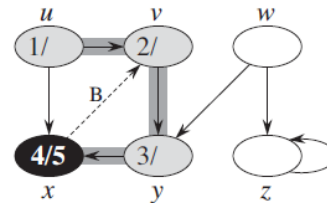
(c)



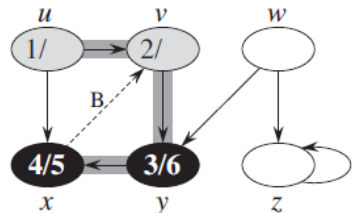
(d)



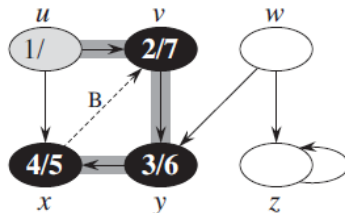
(e)



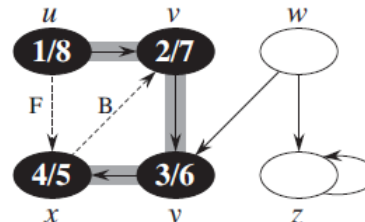
(f)



(g)



(h)



(j)

DFS(G)

```

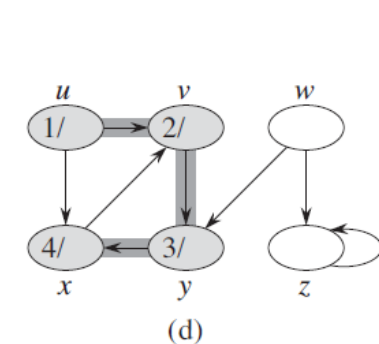
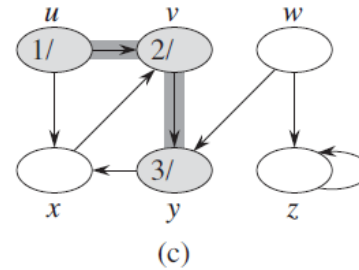
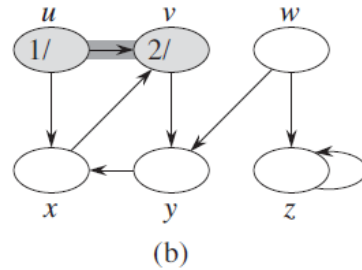
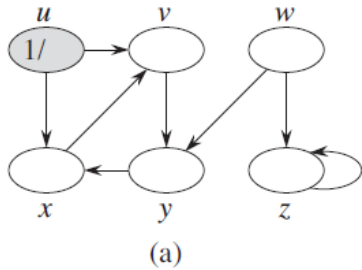
1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
    
```

DFS-VISIT(G, u)

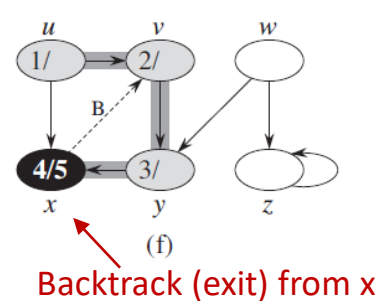
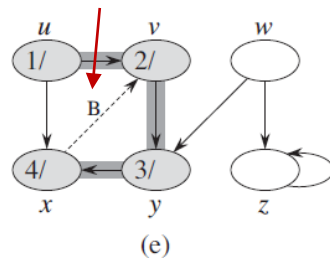
```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
    
```

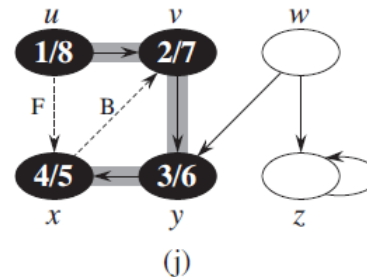
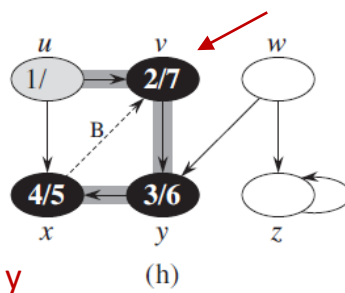
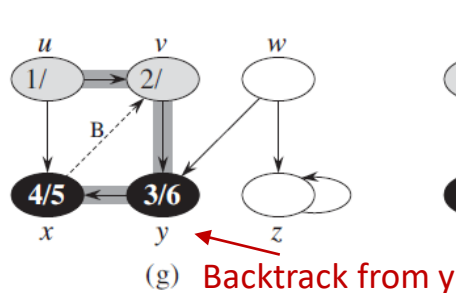
DFS code and example



Bump on a wall 撞墙 deadend.



Backtrack 撤回 from v



DFS(G)

```

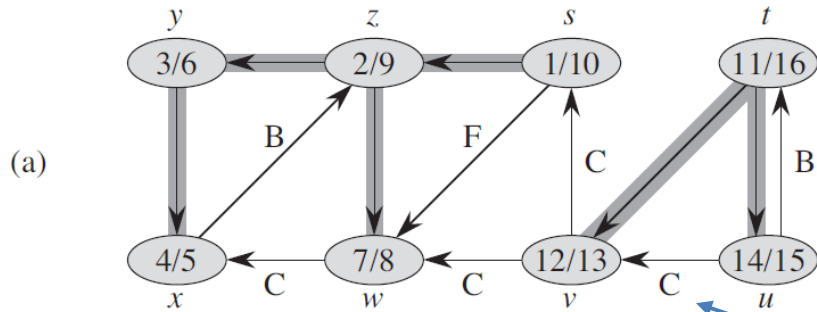
1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
    
```

DFS-VISIT(G, u)

```

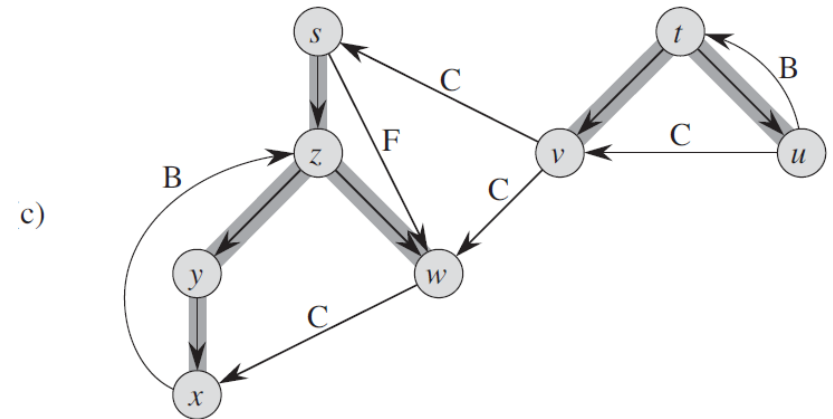
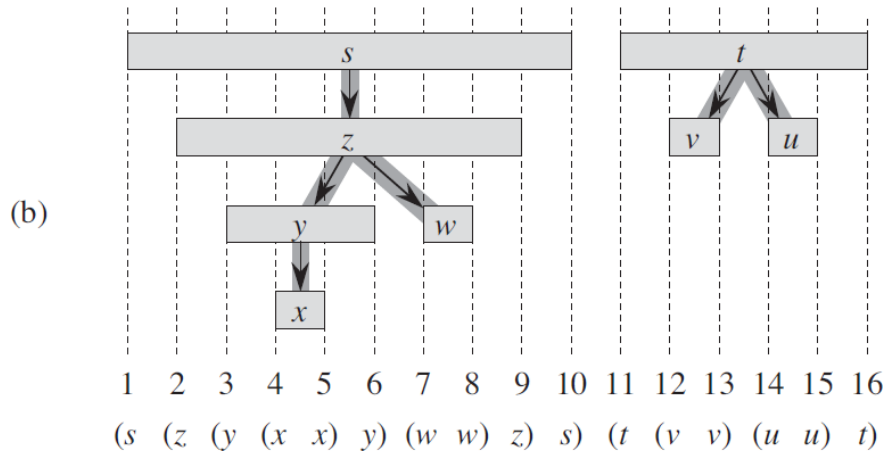
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
    
```

DFS Tree is very useful.



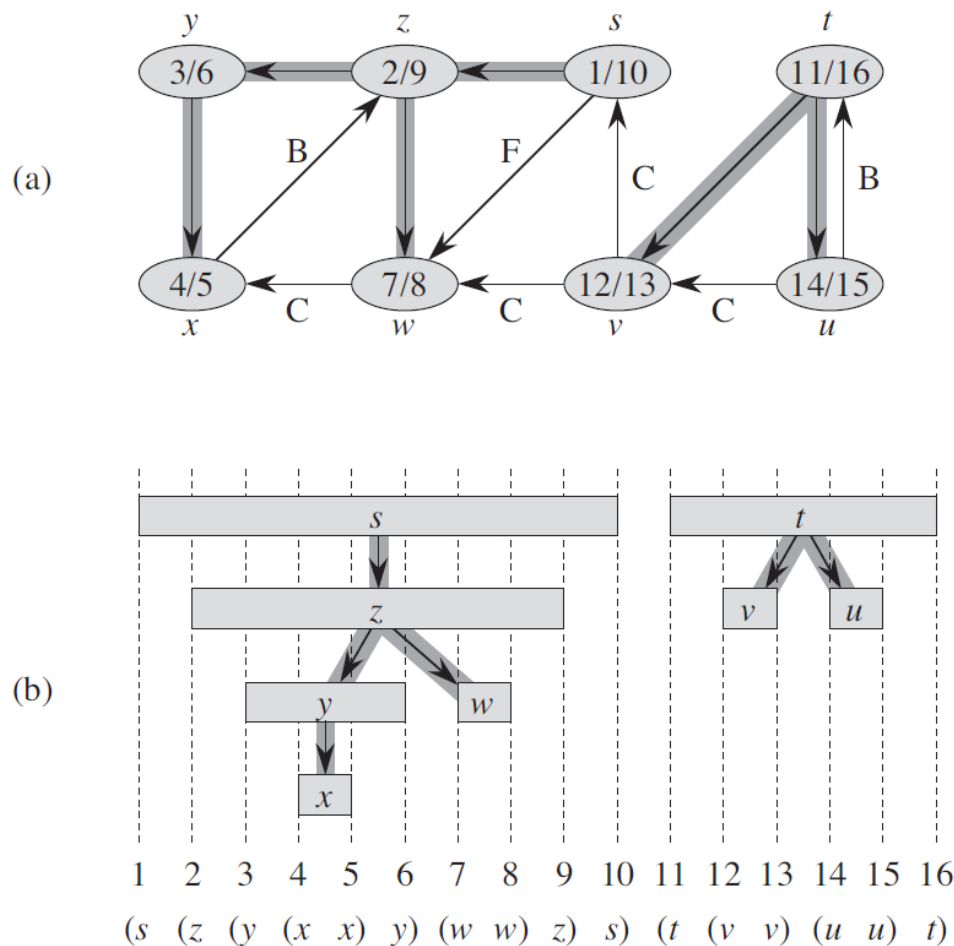
1. **Tree edges** are edges in the depth-first forest G_π . Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v) .
2. **Back edges** are those edges (u, v) connecting a vertex u to an ancestor v in a depth-first tree. We consider self-loops, which may occur in directed graphs, to be back edges.
3. **Forward edges** are those nontree edges (u, v) connecting a vertex u to a descendant v in a depth-first tree.
4. **Cross edges** are all other edges. They can go between vertices in the same depth-first tree, as long as one vertex is not an ancestor of the other, or they can go between vertices in different depth-first trees.

A forest of trees



Parenthesis structure

Figure 22.5 Properties of depth-first search. **(a)** The result of a depth-first search of a directed graph. Vertices are timestamped and edge types are indicated as in Figure 22.4. **(b)** Intervals for the discovery time and finishing time of each vertex correspond to the parenthesization shown. Each rectangle spans the interval given by the discovery and finishing times of the corresponding vertex. Only tree edges are shown. If two intervals overlap, then one is nested within the other, and the vertex corresponding to the smaller interval is a descendant of the vertex corresponding to the larger. **(c)** The graph of part (a) redrawn with all tree and forward edges going down within a depth-first tree and all back edges going up from a descendant to an ancestor.



DFS Tree is very useful.

Mark every entering and exiting of DFS-visit() on the DFS tree.

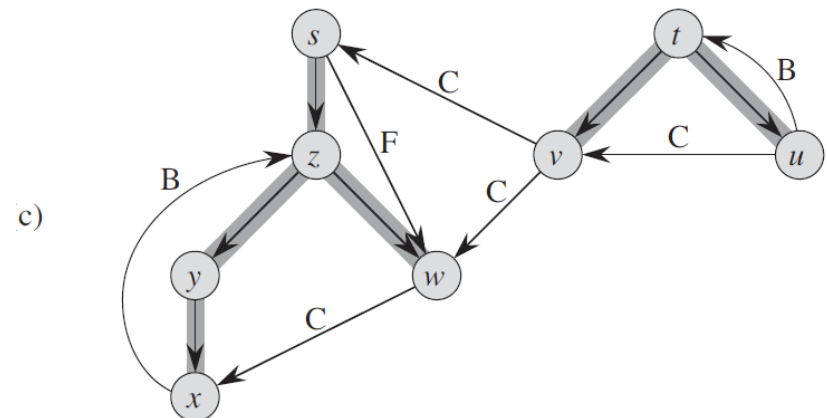


Figure 22.5 Properties of depth-first search. (a) The result of a depth-first search of a directed graph. Vertices are timestamped and edge types are indicated as in Figure 22.4. (b) Intervals for the discovery time and finishing time of each vertex correspond to the parenthesization shown. Each rectangle spans the interval given by the discovery and finishing times of the corresponding vertex. Only tree edges are shown. If two intervals overlap, then one is nested within the other, and the vertex corresponding to the smaller interval is a descendant of the vertex corresponding to the larger. (c) The graph of part (a) redrawn with all tree and forward edges going down within a depth-first tree and all back edges going up from a descendant to an ancestor.

DFS: Recursive calls to DFS-VISIT()

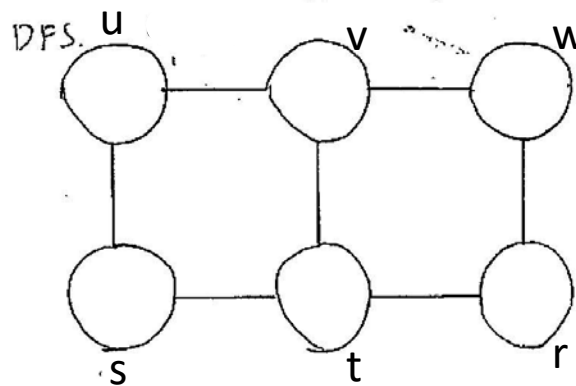
Homework:DFS1

1. Draw the DFS search tree on this graph.

Question: Vertex 1 has two adjacent vertices. Who will be visited first?

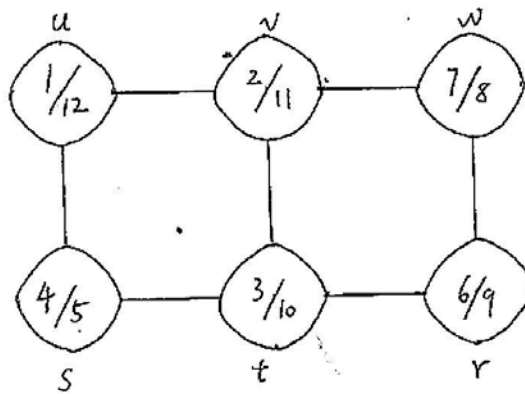
Answer: Depends on the order links in the adjacency list.

DFS: Recursive calls to DFS-VISIT()



Graph Representation
edgelist (undirected)

- (u v)
- (u s)
- (v t)
- (v w)
- (s t)
- (t r)
- (w r)



```

1. DFS-VISIT(u)
2.   DFS-VISIT(v)
3.     DFS-VISIT(t)
4.       DFS-VISIT(s)
5.         EXIT(s)
6.       DFS-VISIT(r)
7.         DFS-VISIT(w)
8.           EXIT(w)
9.         EXIT(r)
10.      EXIT(t)
11.    EXIT(v)
12.  EXIT(u)
    
```

DFS: Recursive calls to DFS-VISIT()

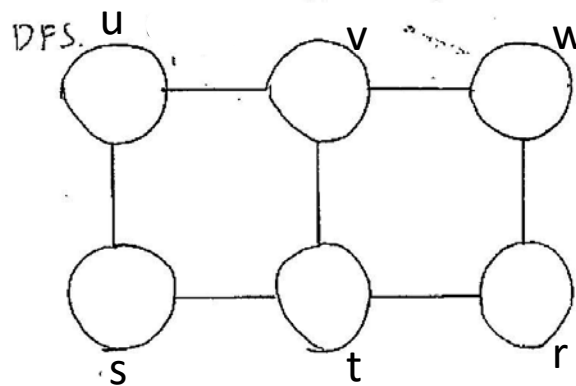
Homework:DFS1

1. Draw the DFS search tree on this graph.

Question: Vertex 1 has two adjacent vertices. Who will be visited first?

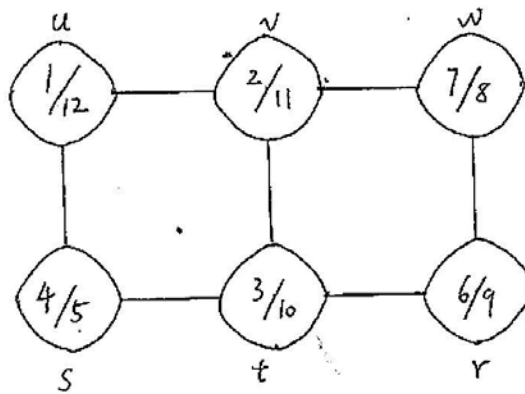
Answer: Depends on the order links in the adjacency list.

DFS: Recursive calls to DFS-VISIT()



Graph Representation
edgelist (undirected)

- (u v)
- (u s)
- (v t)
- (v w)
- (s t)
- (t r)
- (w r)

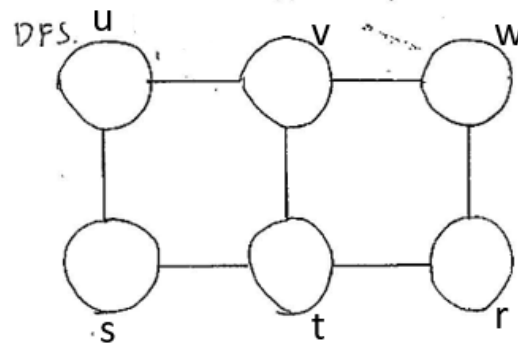


```

1. DFS-VISIT(u)
2.   DFS-VISIT(v)
3.     DFS-VISIT(t)
4.       DFS-VISIT(s)
5.         EXIT(s)
6.       DFS-VISIT(r)
7.         DFS-VISIT(w)
8.           EXIT(w)
9.         EXIT(r)
10.      EXIT(t)
11.    EXIT(v)
12.  EXIT(u)
    
```

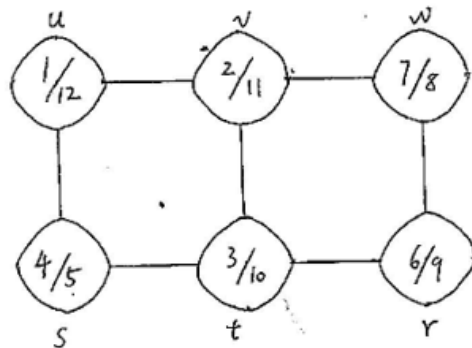
DFS does Backtracking.

DFS: Recursive calls to DFS-VISIT()



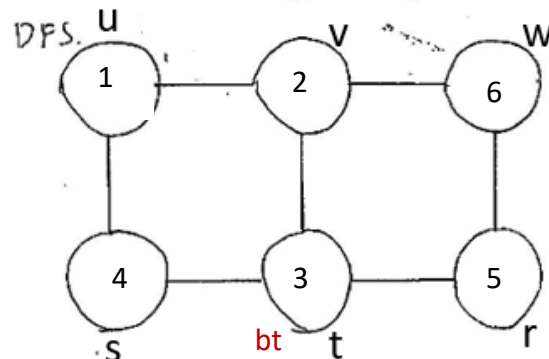
Graph Representation
edgelist (undirected)

- (u v)
- (u s)
- (v t)
- (v w)
- (s t)
- (t r)
- (w r)



```

1. DFS-VISIT(u)
2.   DFS-VISIT(v)
3.     DFS-VISIT(t)
4.       DFS-VISIT(s)
5.         EXIT(s)
6.       DFS-VISIT(v)
7.         DFS-VISIT(w)
8.           EXIT(w)
9.         EXIT(v)
10.      EXIT(t)
11.    EXIT(v)
12.  EXIT(u)
  
```



More clear
mark on DFS
visit. Finish
time are
ignored.

The backtrack steps are implicit,
finish time are not marked. In computer
implementation, backtrack is done one
edge at a time, finish time do exist.

Backtrack from node s to node t (marked bt), visit another
neighbor of t, which is r, and continue.

P1. DFS backtrack. Find a path from start node to goal node.

Right figure shows a DFS visit route. It hits a deadend at (3,2).

Your task:

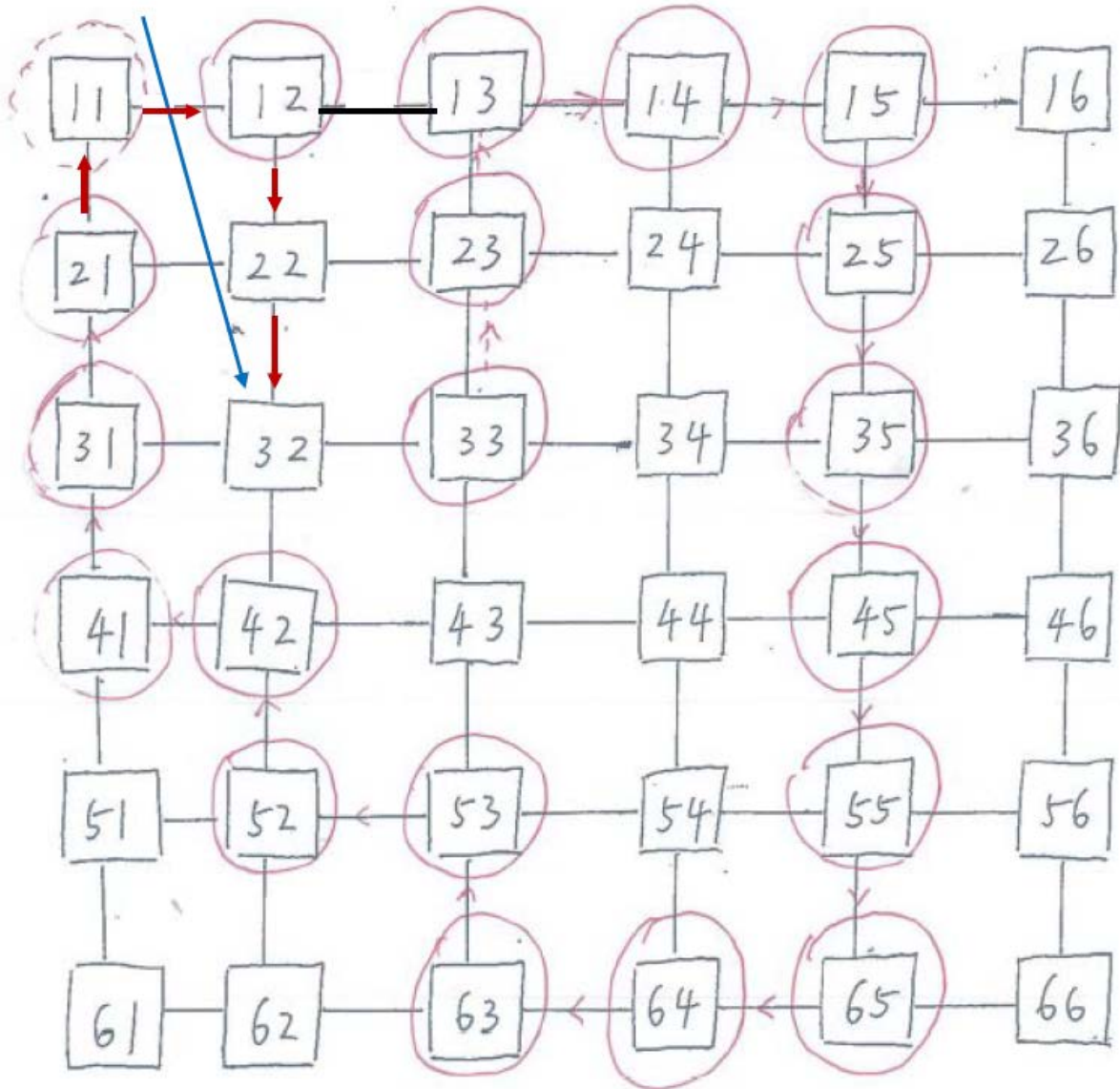
(a) Continue DFS. Do the backtrack and find the first **backtrack node** where DFS can visit thus starting another route. Mark this node as **bf**.

(b) If your path reach the goal, stop. Otherwise do backtrack and find the 2nd backtrack node, mark it. Follow step (a).

Repeat, until you find a path that hit the goal.

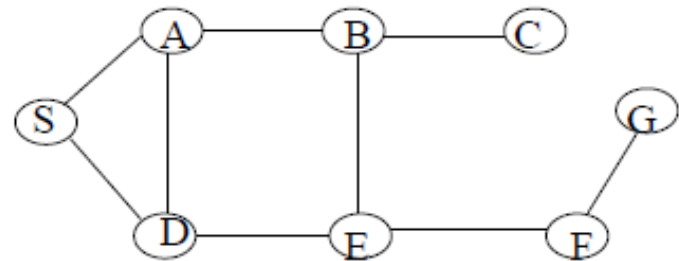
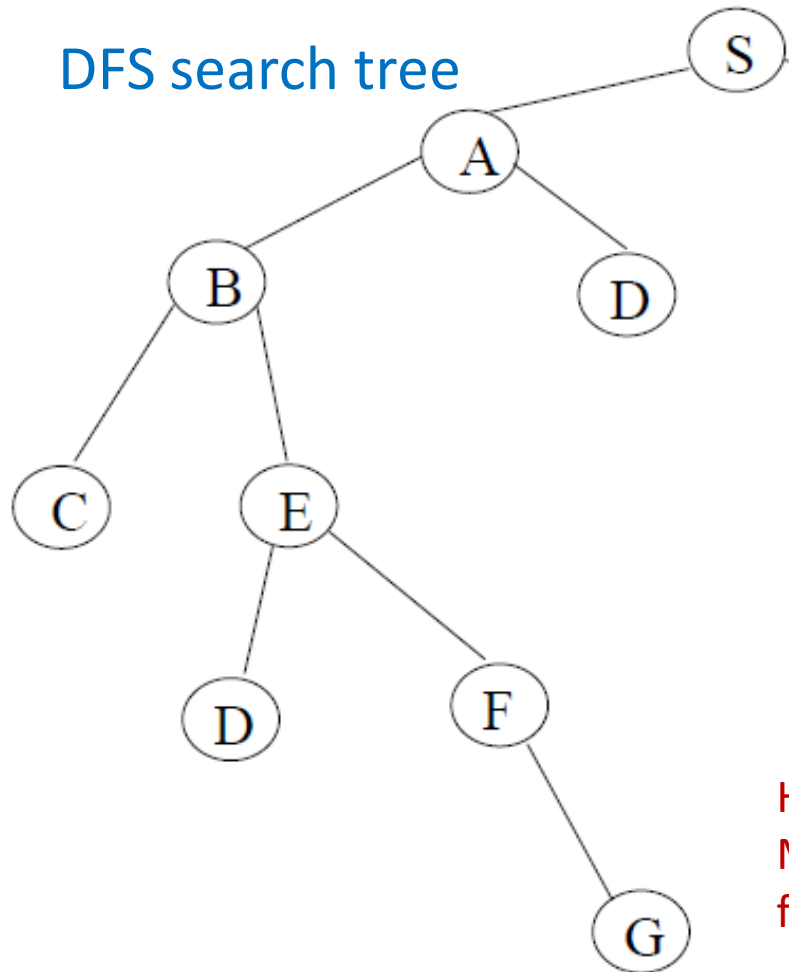
Start (3,3) $\rightarrow$ Goal (5,4)

DFS hits a dead-end



Depth-First Search (DFS)

DFS search tree



Homework:DFS2

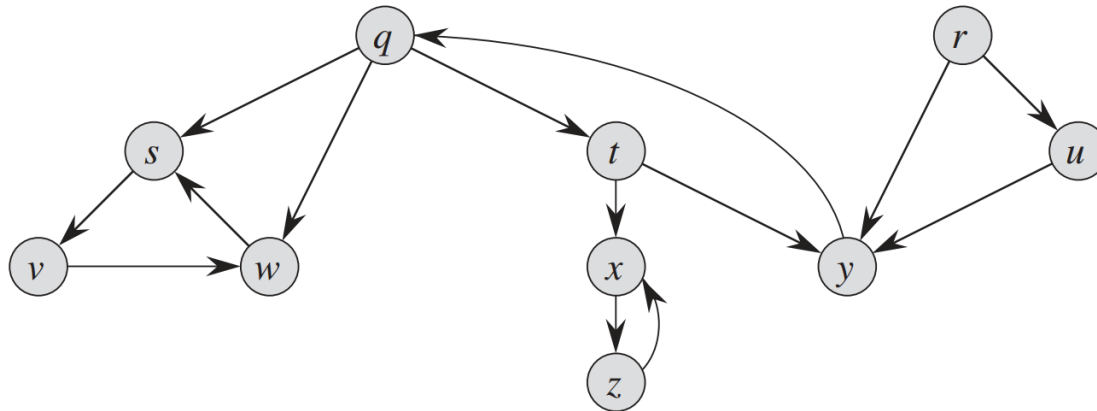
Mark the entering and leave time for every nodes.

Homework: DFS3

Show how DFS visit all nodes in the following graph.

Assume: In DFS(), the nodes are visited alphabetically.

Assume: The adjacency nodes of each nodes are listed alphabetically.



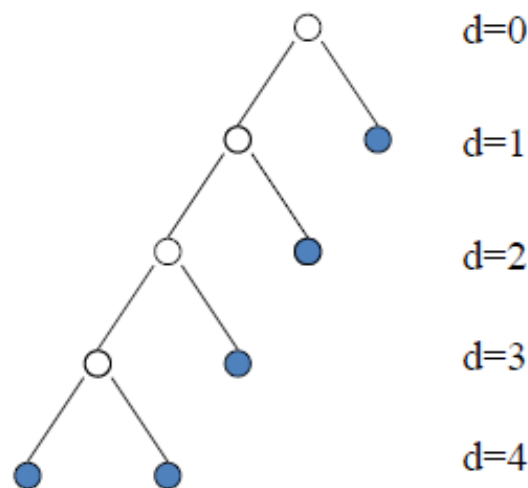
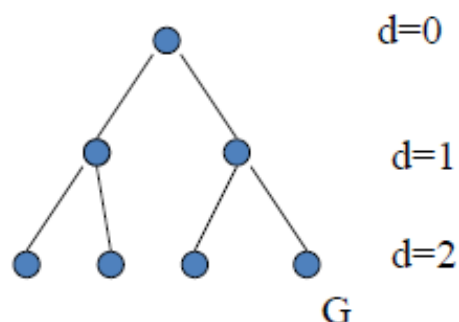
Complexity of Depth-First Search?

- Time Complexity
 - assume d is deepest path in the search space
 - assume (worst case) that there is 1 goal leaf at the RHS
 - so DFS will expand all nodes

$$= 1 + b + b^2 + \dots + b^d$$

$$= O(b^d)$$

- Space Complexity (for tree-search)
 - how many nodes can be in the queue (worst-case)?
 - **$O(bd)$ if deepest node at depth d**

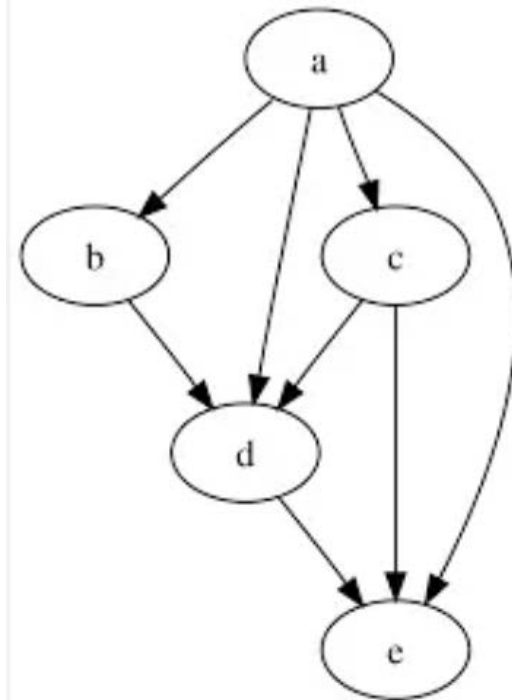
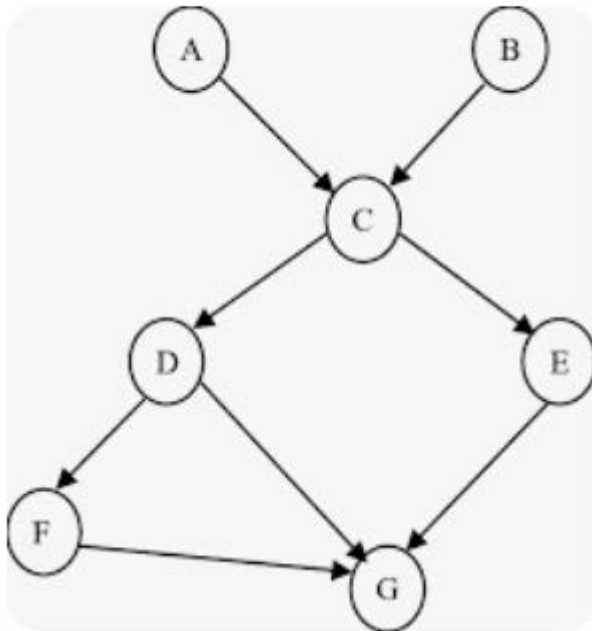


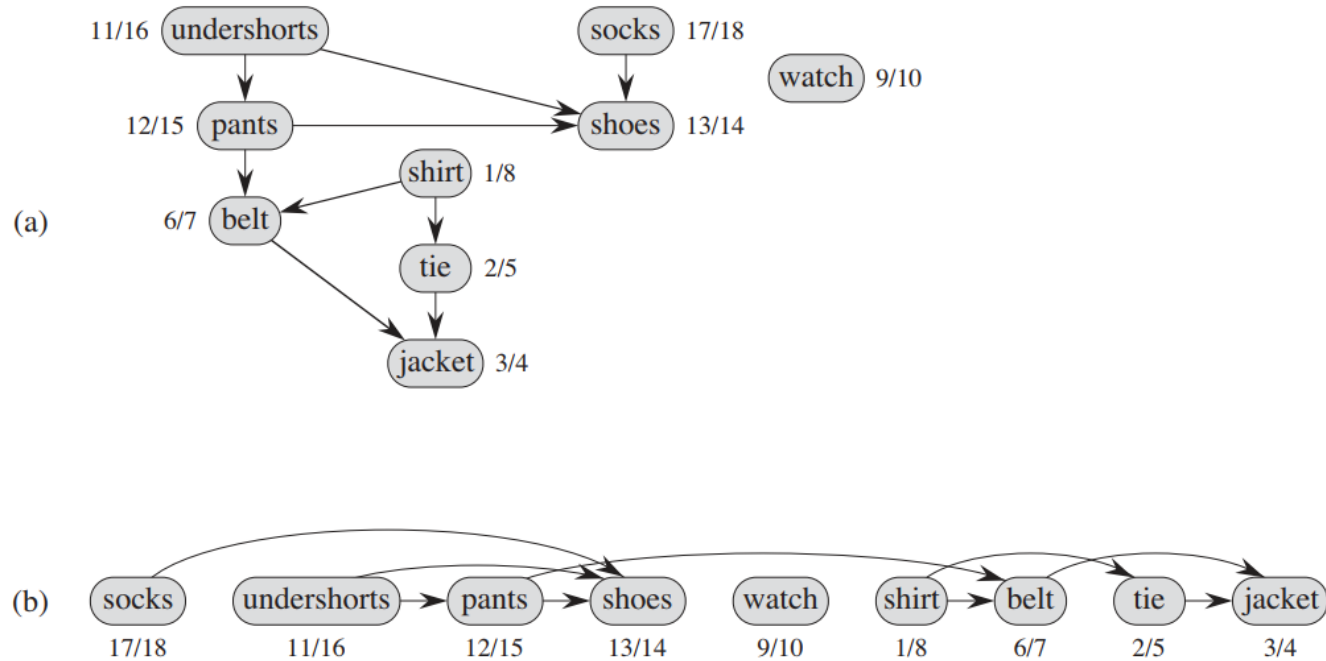
Directed Acyclic Graph (DAG) is one of the most widely used graphs

No cycles in the graph

Most important properties: topological ordering

Directed acyclic graph - Wikiwa...



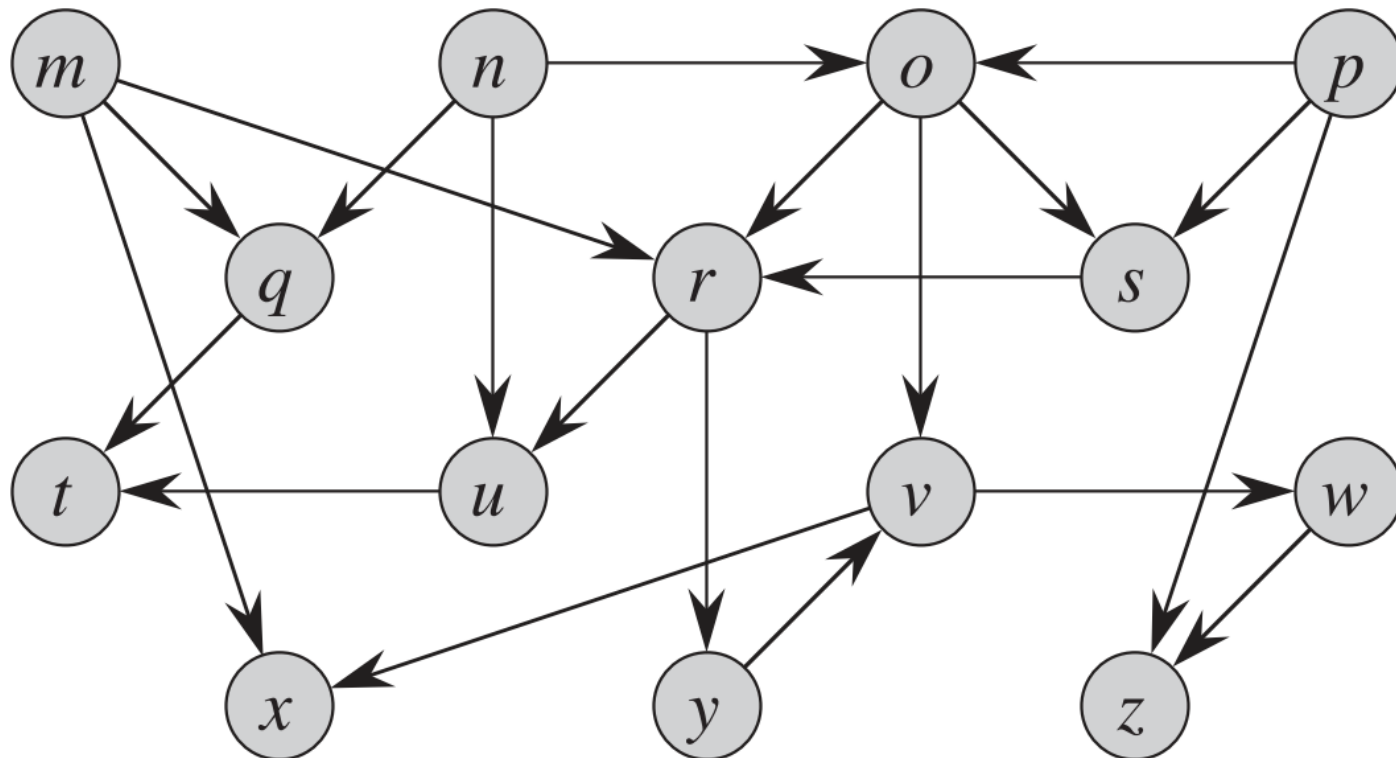


TOPOLOGICAL-SORT(G)

- 1 call $\text{DFS}(G)$ to compute finishing times $v.f$ for each vertex v
- 2 as each vertex is finished, insert it onto the front of a linked list
- 3 **return** the linked list of vertices

Homework: DFS4:

Finding the topological sort/ordering of the following graph



Depth-First tree-search Properties

- Non-optimal solution path
- Incomplete unless there is a depth bound
- (we will assume depth-limited DF-search)
- Re-expansion of nodes (when the state-space is a graph)
- Exponential time
- Linear space (for tree-search)

DFS is very useful

DFS on directed acyclic graph(DAG) can discover topological order

DFS on directed graph can discover strongly connected components

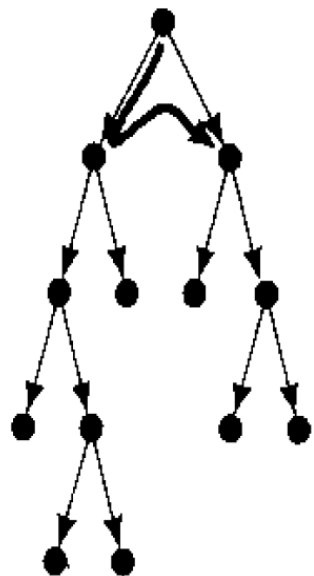
In AI and robotics, DFS is improved into Iterative Deepening DFS

Iterative Deepening DFS (IDDFS)

By explore DFS on the 2-digit locker graph, you can see:

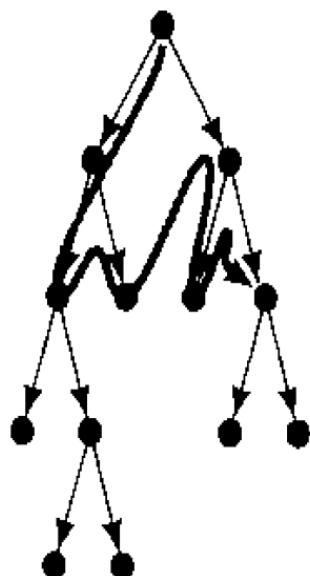
- BFS is very ordered,
- but DFS can go any direction, in a uncontrollable way

In real-life applications of AI, Robotics, we control the depth of DFS, leading to Iterative Deepening DFS [Richard Korf, 1985]

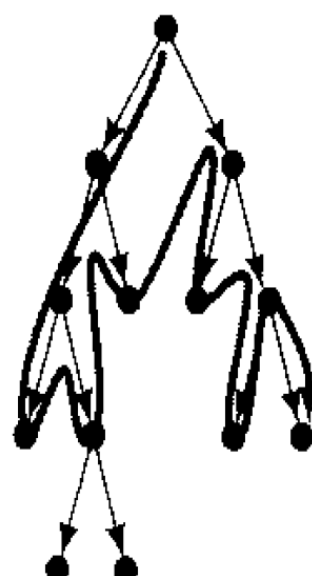


Depth bound = 1

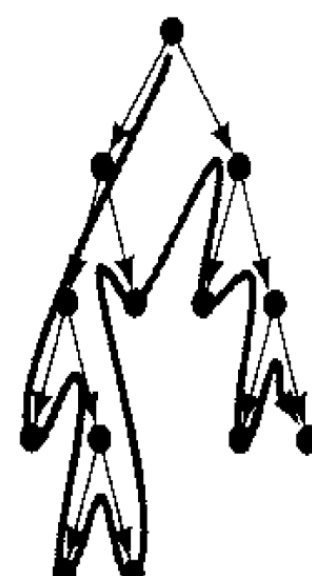
IDDSF=BFS



Depth bound = 2



Depth bound = 3

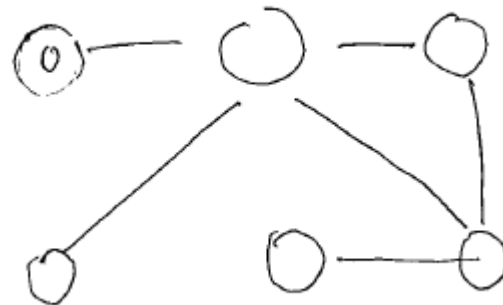
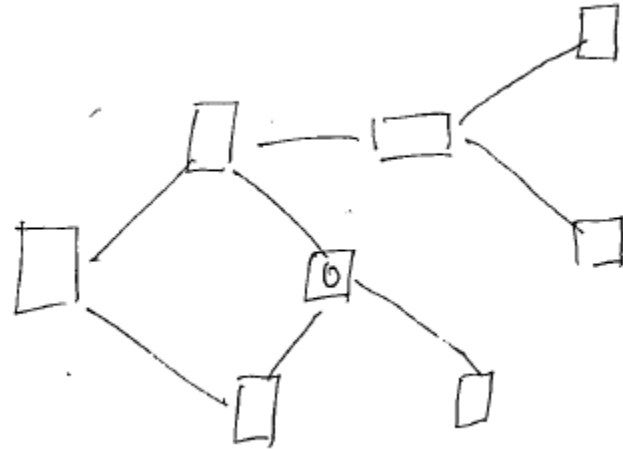


Depth bound = 4

IDDSF=DFS

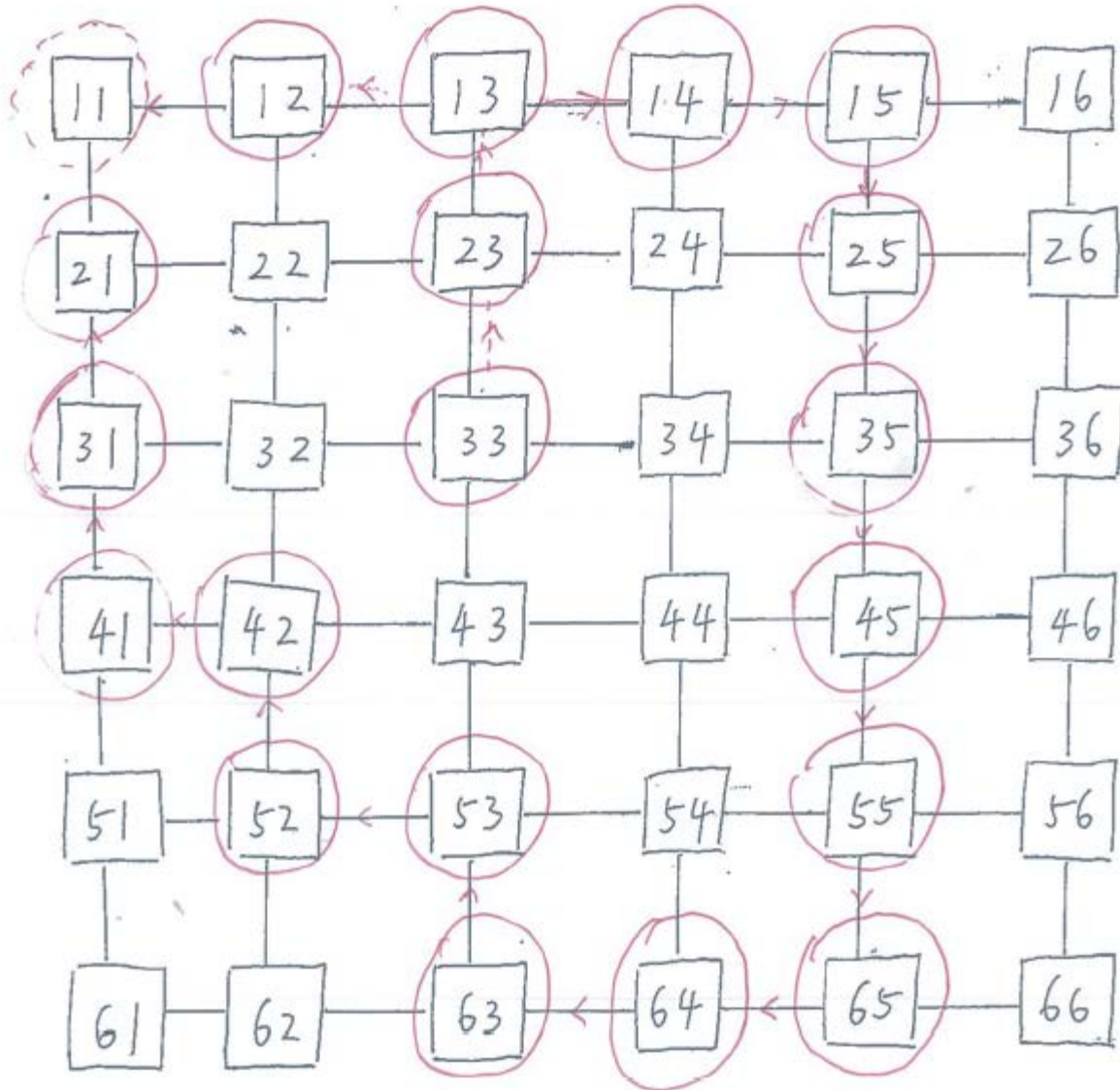
Homework DFS 2.

Do BSF and DFS on the two graphs on the left. Starting from the node denoted with "0". Draw corresponding BSF trees and DFS trees.



DFS goes uncontrollably,
miss goal when it is very
close to it. BFS finds the
goal faster.

Start (3,3) $\rightarrow$ Goal (5,4)



Class programming homework/project (10% of final grade)

1. Implement a depth-limited DFS algorithm.
2. Implement an iterative deepening algorithm.
3. Design a travel suitcase locker combination graph (3 digit, 0-9)
4. Assume the true combination is 123. You start from a randomly choosing combination, search the correct combination. Repeat this for 10 times.
5. Run BFS, DFS and Iterative Deepening DFS, doing task (4) , recording average search steps.
6. Write a 3 page report. Submit code.
7. Due Oct 10.

Uniform Search – Dijkstra Algorithm

(invented in 1956 by Edsger Dijkstra)

Find the shortest paths from a starting node to all other nodes on a weighted graph.

It greedily expands a set S containing nodes according to the distances to starting node.

In computer science, **greedy** means intuitively best (most obvious) choice at the moment.

Uniform Search: Dijkstra Algorithm

From "Introduction to Algorithms"

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.\text{Adj}[u]$ 
8          RELAX( $u, v, w$ )
    
```

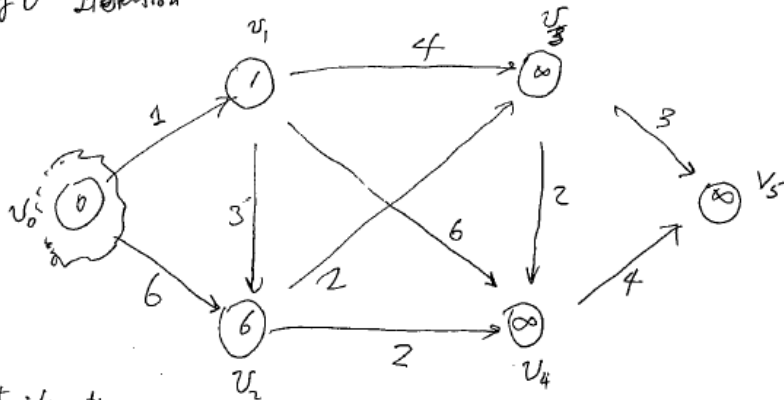
update

RELAX(u, v, w)

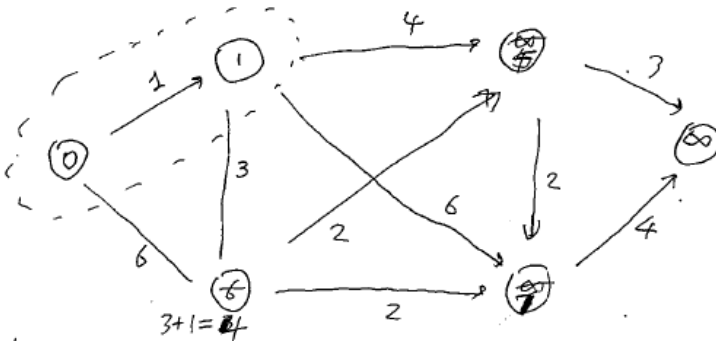
```

1  if  $v.d > u.d + w(u, v)$ 
2       $v.d = u.d + w(u, v)$ 
3       $v.\pi = u$ 
    
```

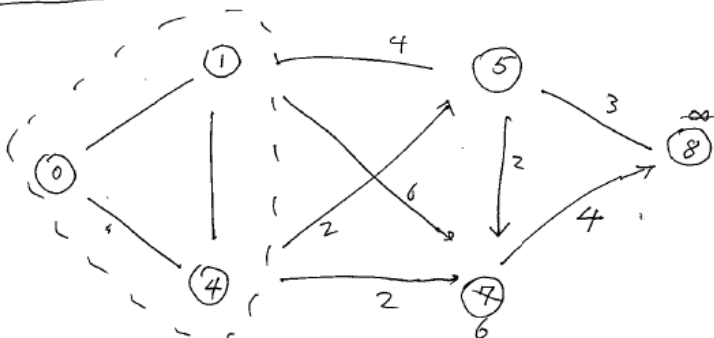
End of 0th Iteration



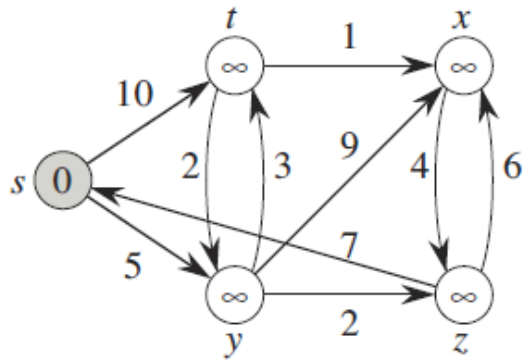
End of 1st iteration



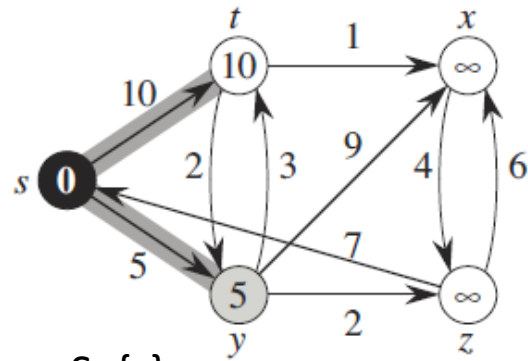
end of 2nd iteration



Dijkstra Algorithm Example

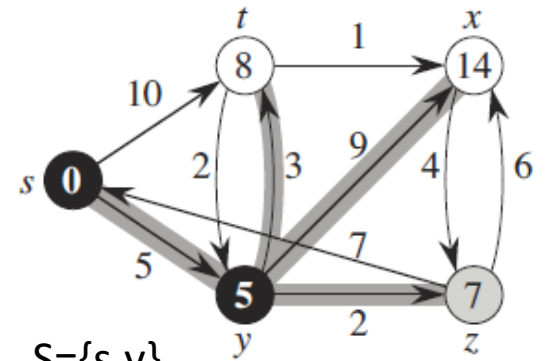


(a)



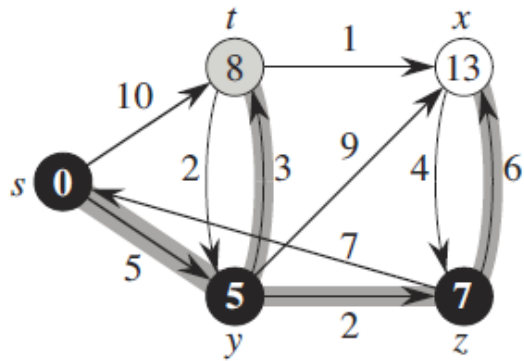
$S=\{s\}$

(b)



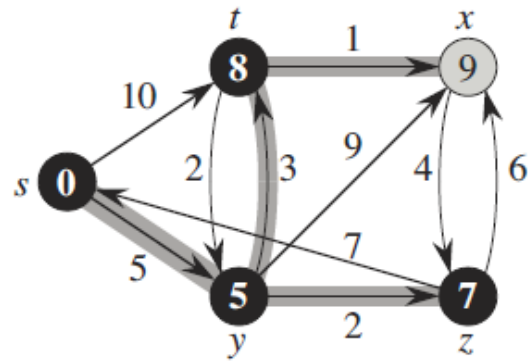
$S=\{s, y\}$

(c)



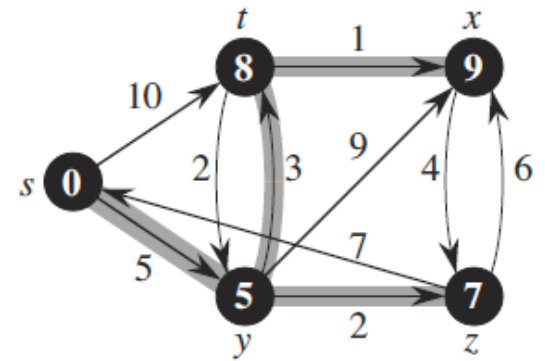
(d)

$S=\{s, y, z\}$



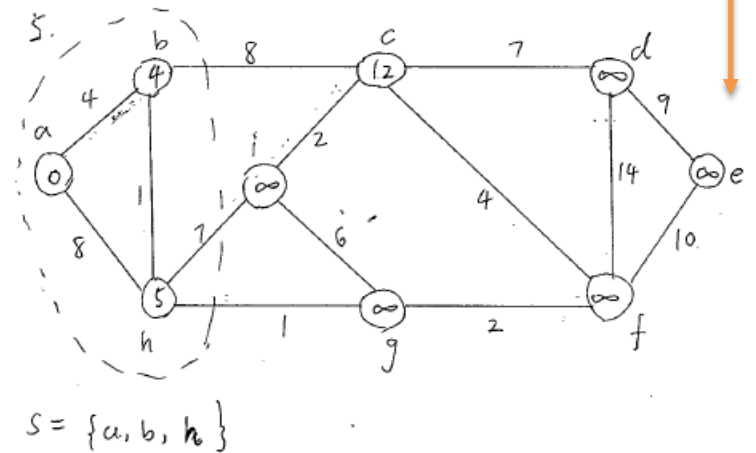
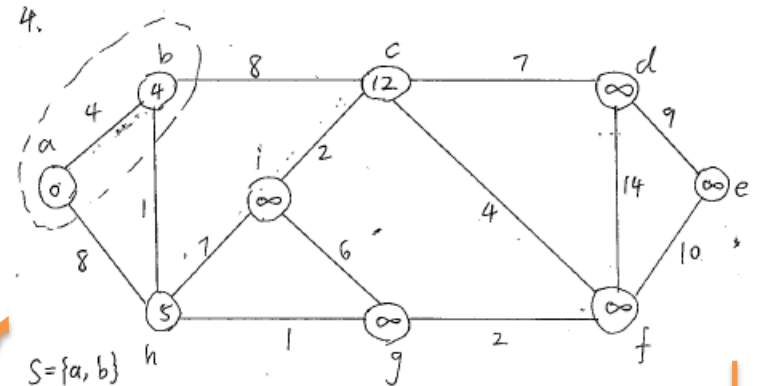
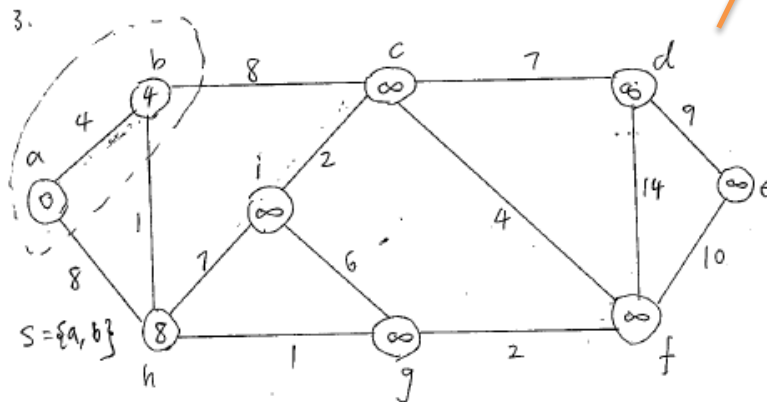
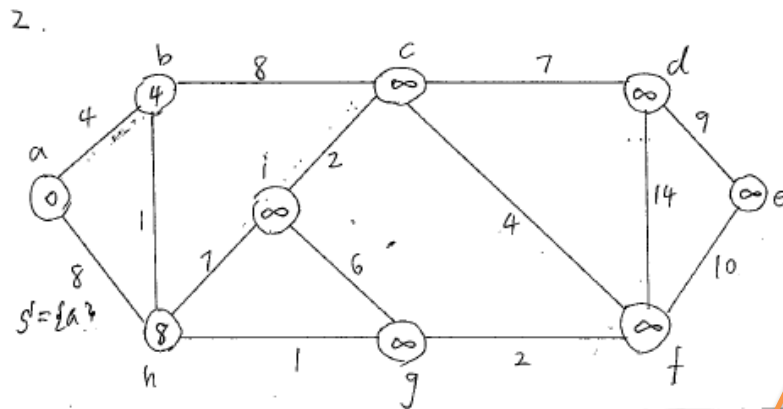
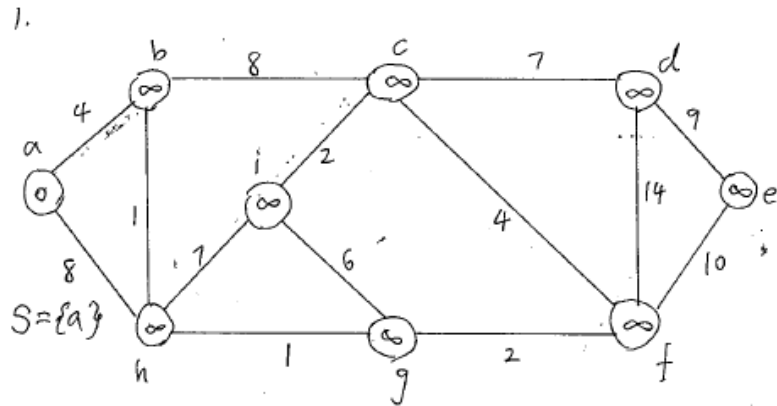
(e)

$S=\{s, y, z, t\}$



(f)

Another Example of Dijkstra Algorithm



HOMEWORK: Dijkstra1.
Complete the algorithm on this graph.

Uniform Cost Search

- Completeness:** • Guaranteed to find optimal solution (as long as all steps have >0 cost)
- When a node is selected for expansion, a shortest path to it has been found
- Optimality:** • UCS expands in the order of optimal path cost

Proof of Dijkstra Algorithm Optimality.

Key step is the greedy expansion of the set S . Every expansion chooses the closest node to include. This guarantees the optimum of the nodes distance from the start.

Key observation: greedy algorithm, every step is chosen from all possible choices.

